



Concevoir et analyser une base de données NoSQL

"... dans la perspective d'un rapport qui doit étayer l'effet "Jeux Olympiques 2024" sur l'offre de logements à Paris ..."

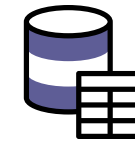


Pascal DUVAL

AI/ML Engineer

Restauration et inspection d'une base MongoDB

Processus, structure des documents et avantages NoSQL



Restauration d'une base de données sur MongoDB

Nous partirons d'une sauvegarde csv que nous importerons dans MongoDB Compass



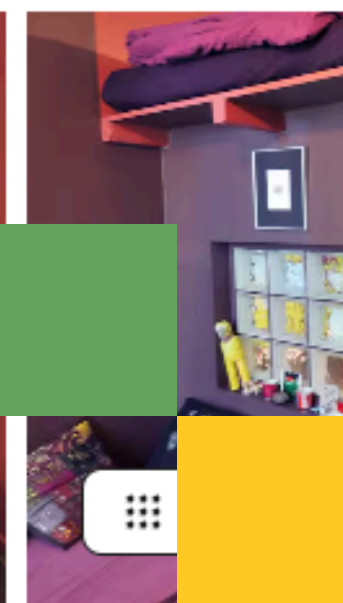
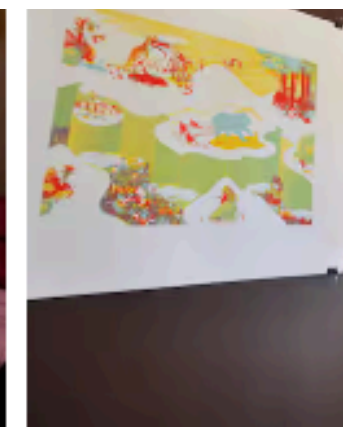
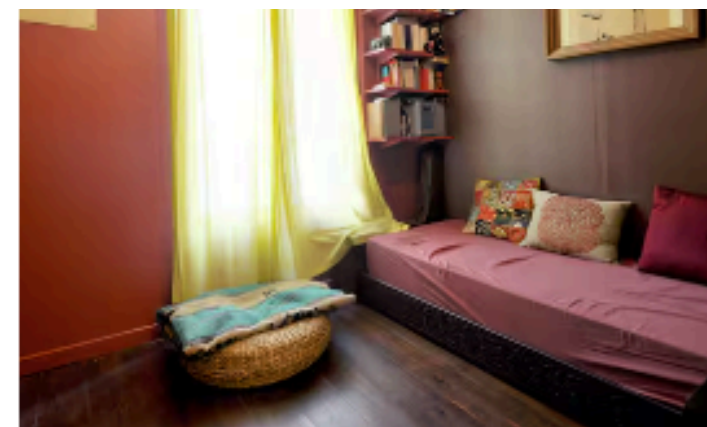
Observation de la structure des documents et des types de données

Analyse de schéma et types présents dans les documents.



Explication des avantages du NoSQL dans ce contexte de données.

Points forts de NoSQL pour ce jeux de données.



Le Fichier De Sauvegarde Des Offres Airbnb À Paris

Comment a-t-il été obtenu ?

1. **Scraping** (HTML / JSON) de pages publiques sur Airbnb (non via API)
2. Agrégation des données dans un fichier csv (**listings_Paris.csv**)
3. Extraction régulière (ex: tous les mois) -> **last_scraped** (datetime)
4. Formatage selon une structure standardisée (cf. **Data+Dictionary.xlsx**).



Évaluer L'impact Des Locations Airbnb Sur Le Marché Du Logement

Comment s'en servir ?

- Mesurer la part de logements entiers en location récurrente (`room_type == "Entire home/apt"` & `availability_[30-60-365]`) . Un croisement avec `neighbourhood_cleansed` permettriy si l'on disposait d'une base de données de locations traditionnelles d'**estimer le nombre de logements retirés du marché**.
- Identifier le nombre de multi-loueurs professionnels - 3 logements actifs par hôte → signal d'activité commerciale.
- Cartographier la concentration géographique. on dispose de latitude & longitude. Avec une cartographie (type Folium, GeoPandas), on peut repérer les quartiers les plus touchés (hypercentre, zones touristiques) / faire des heatmaps de densité Airbnb

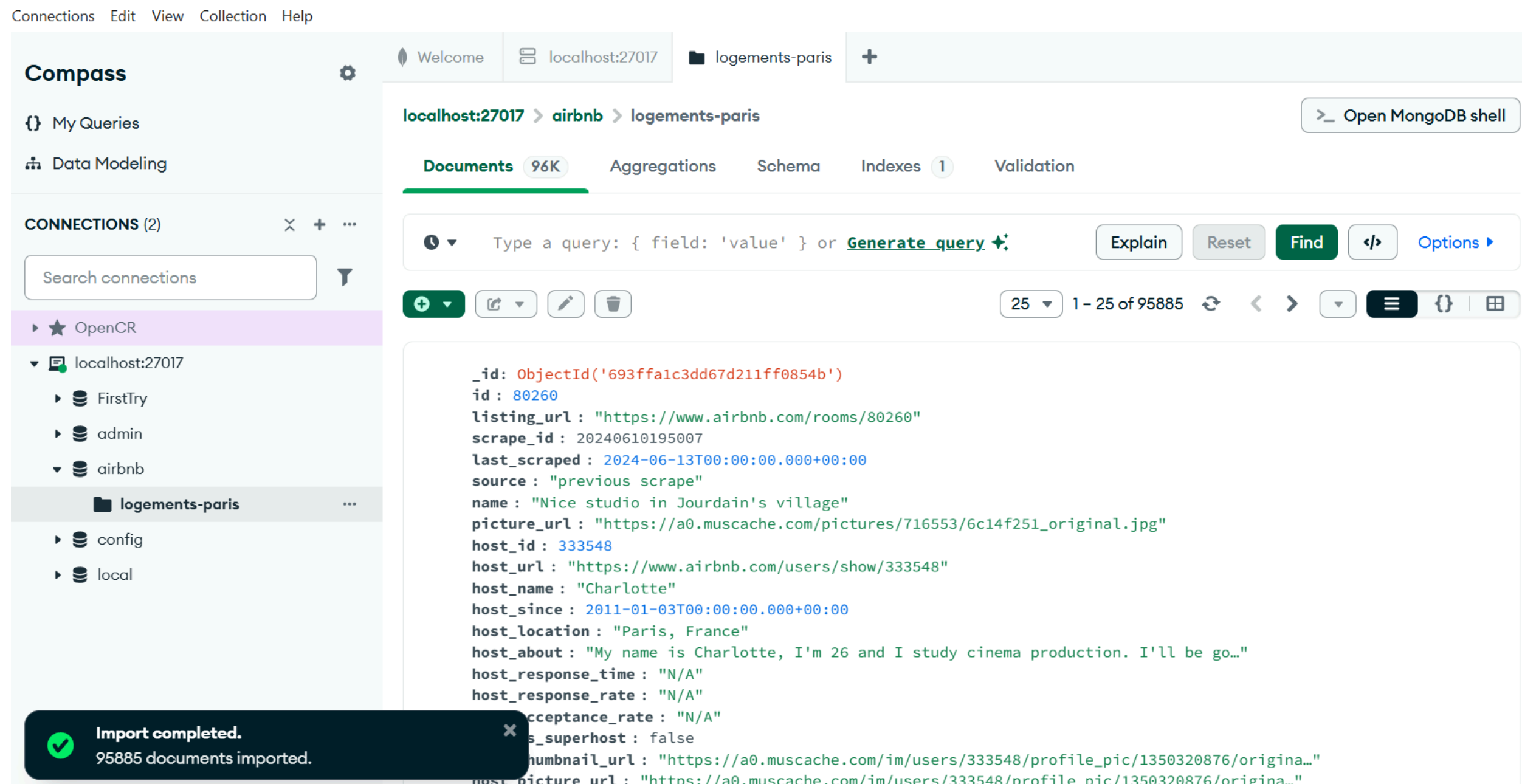
MongoDB Compass

Database name : airbnb

Collection : logements-paris

Storage Size : 70.82 MB

95885 documents



The screenshot displays the MongoDB Compass web interface. On the left sidebar, the 'Connections' panel shows a list of connections, including 'localhost:27017' and 'airbnb'. The 'logements-paris' collection is selected. The main panel shows the 'Documents' tab with a count of 95885. A query bar at the top of the document list contains the text 'Type a query: { field: 'value' } or Generate query'. Below the query bar, a document is displayed in a JSON format. The document contains fields such as '_id', 'id', 'listing_url', 'scrape_id', 'last_scraped', 'source', 'name', 'picture_url', 'host_id', 'host_url', 'host_name', 'host_since', 'host_location', 'host_about', 'host_response_time', 'host_response_rate', 'acceptance_rate', 'is_superhost', 'thumbnail_url', and 'host_picture_url'. A notification at the bottom left of the interface states 'Import completed. 95885 documents imported.'

Connections Edit View Collection Help

Compass

{ } My Queries

Data Modeling

CONNECTIONS (2)

Search connections

★ OpenCR

localhost:27017

FirstTry

admin

airbnb

logements-paris

config

local

Welcome localhost:27017 logements-paris

localhost:27017 > airbnb > logements-paris

Documents 96K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or Generate query

Explain Reset Find Options

25 1 - 25 of 95885

```
{
  "_id": ObjectId('693ffa1c3dd67d211ff0854b'),
  "id": 80260,
  "listing_url": "https://www.airbnb.com/rooms/80260",
  "scrape_id": 20240610195007,
  "last_scraped": "2024-06-13T00:00:00.000+00:00",
  "source": "previous scrape",
  "name": "Nice studio in Jourdain's village",
  "picture_url": "https://a0.muscache.com/pictures/716553/6c14f251_original.jpg",
  "host_id": 333548,
  "host_url": "https://www.airbnb.com/users/show/333548",
  "host_name": "Charlotte",
  "host_since": "2011-01-03T00:00:00.000+00:00",
  "host_location": "Paris, France",
  "host_about": "My name is Charlotte, I'm 26 and I study cinema production. I'll be go...",
  "host_response_time": "N/A",
  "host_response_rate": "N/A",
  "acceptance_rate": "N/A",
  "is_superhost": false,
  "thumbnail_url": "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/origina...",
  "host_picture_url": "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/origina..."
}
```

Import completed.
95885 documents imported.

```
{
  "_id": {
    "$oid": "693ffa1c3dd67d211ff0854b"
  },
  "id": 80260,
  "listing_url": "https://www.airbnb.com/rooms/80260",
  "scrape_id": {
    "$numberLong": "20240610195007"
  },
  "last_scraped": {
    "$date": "2024-06-13T00:00:00.000Z"
  },
  "source": "previous scrape",
  "name": "Nice studio in loundain's village",
  "picture_url": "https://a0.muscache.com/pictures/716553/6c14f251_original.jpg",
  "host_id": 333548,
  "host_url": "https://www.airbnb.com/users/show/333548",
  "host_name": "Charlotte",
  "host_since": {
    "$date": "2011-01-03T00:00:00.000Z"
  },
  "host_location": "Paris, France",
  "host_about": "My name is Charlotte, I'm 26 and I study cinema production. I'll be gone the whole year to study in Grenoble and that's my father Jean Luc, who will keep an eye on my little studio and take care of you. \r\n",
  "host_response_time": "N/A",
  "host_response_rate": "N/A",
  "host_acceptance_rate": "N/A",
  "host_is_superhost": false,
  "host_thumbnail_url": "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/original.jpg?aki_policy=profile_small",
  "host_picture_url": "https://a0.muscache.com/im/users/333548/profile_pic/1350320876/original.jpg?aki_policy=profile_x_medium",
  "host_neighbourhood": "Buttes-Chaumont - Belleville",
  "host_listings_count": 1,
  "host_total_listings_count": 1,
  "host_verifications": "['email', 'phone']",
  "host_has_profile_pic": true,
  "host_identity_verified": true,
  "neighbourhood_cleansed": "Ménilmontant",
  "latitude": 48.87131,
  "longitude": 2.38848,
  "property_type": "Entire rental unit",
  "room_type": "Entire home/apt",
  "accommodates": 3,
  "bathrooms_text": "1 bath",
  "bedrooms": 1,
  "amenities": [
    "Hangers",
    "Essentials",
    "Wifi",
    "Dishes and silverware",
    "Dedicated workspace",
    "Hair dryer",
    "Kitchen".
  ]
}
```

Les documents ...

MongoDB stocke ce document en format BSON, qui est comme du JSON enrichi. On peut donc faire des requêtes normales comme { **host_is_superhost: true** } ou des filtres sur les dates { **last_scraped: { \$gte: new Date("2024-01-01") }** }

une base très complète, bien structurée et prête pour l’analyse qui permet :

- des analyses statistiques (reviews, prix, disponibilité)
- des analyses géographiques (quartiers, densité)
- des analyses de comportement hôte (superhost, multi-annonce, etc.)

Avantages de (MongoDB) NoSQL

Besoin métier	Avantage	Explication
Annonces très diverses	JSON libre et flexible	MongoDB stocke ça nativement en BSON, ce qui permet de travailler directement avec les structures complexes, sans transformation
Scraping fréquent	Schéma évolutif sans migration	MongoDB est schéma-libre, Ce qui est parfait pour un scraping régulier : les données changent avec le temps (Airbnb modifie ses pages, formats, etc.)
Analyse statistique	Aggregation pipeline puissante	Possibilité de faire des groupements, des tris . Le <i>Aggregation Framework</i> de MongoDB remplace dans de nombreux cas des requêtes SQL complexes ou des transformations Pandas.
Hétérogénéité des données	chaque document est autonome et flexible	Toutes les annonces ne contiennent pas exactement les mêmes champs (exempels : présence ou pas de reviews_per_month, picture_url). Avec SQL, tu aurais beaucoup de NULL dans une table à colonnes fixes
Performance	Lecture rapide, requêtes par champ indexé	MongoDB permet d'indexer des champs simples, des champs imbriqués pour par exemple des requêtes ultra rapides par quartier, des calculs de proximité...
Données en volume	Scalabilité horizontale possible	MongoDB peut se répartir sur plusieurs serveurs ("sharding"), ce que les bases SQL traditionnelles gèrent difficilement sans architectures complexes.

"Schéma"

1. Tout est clé/valeur : "clé" : "valeur"
2. Un document est encapsulé dans des accolades {...},
3. Une valeur peut être un type scalaire (entier, nombre, texte, booléen, null), des listes de valeurs [...], ou des documents imbriqués

Schéma ?

Il faut savoir que MongoDB ne gère aucun schéma pour les collections. Aucune structure fixe ni de typage (ce qui fait la force des SGBDR).

C: > Users > karap > OpenClassRooms > dataprojet7 > schema-airbnb-logements-paris-mongoDBJSON.js

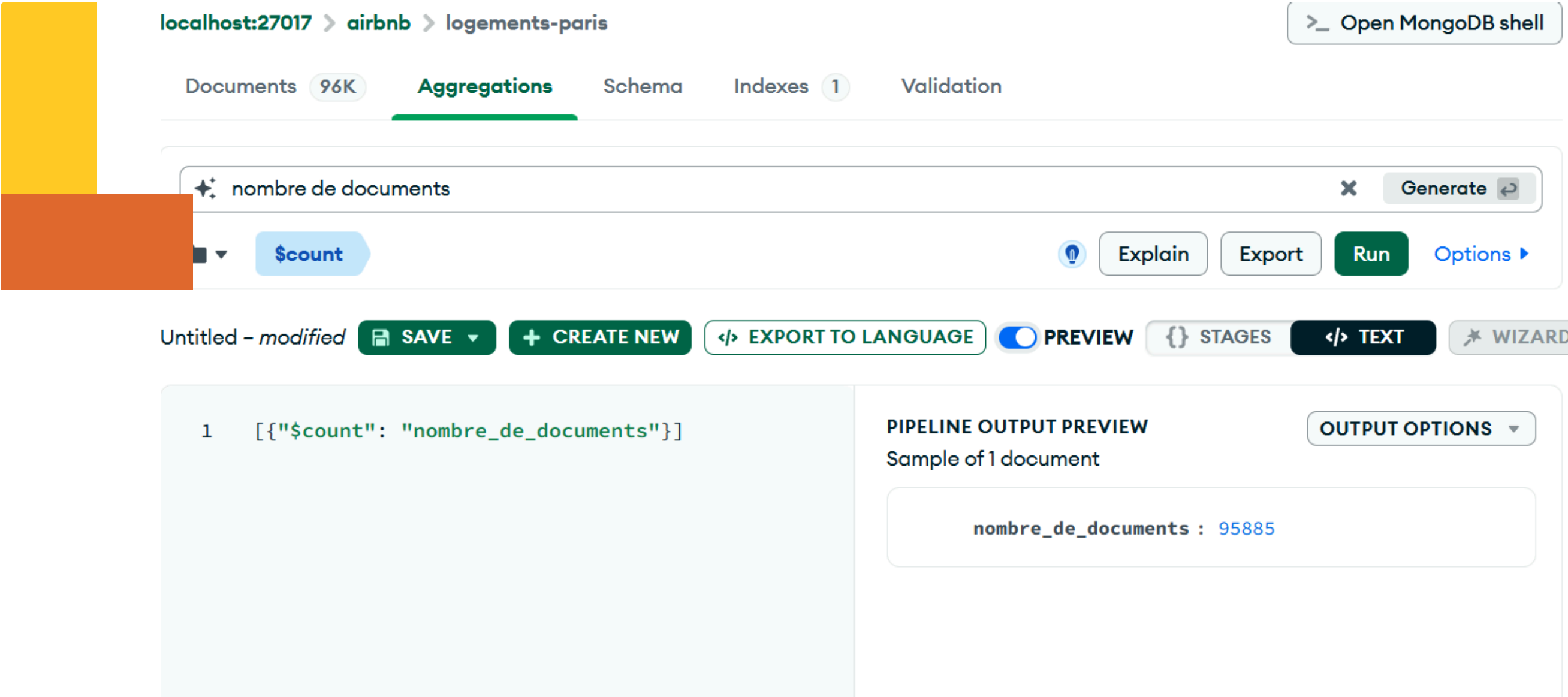
```
1  {
2    "$jsonSchema": {
3      "bsonType": "object",
4      "required": [
5        "_id",
6        "accommodates",
7        "amenities",
8        "availability_30",
9        "availability_365",
10       "availability_60",
11       "availability_90",
12       "calculated_host_listings_count",
13       "calculated_host_listings_count_entire_homes",
14       "calculated_host_listings_count_private_rooms",
15       "calculated_host_listings_count_shared_rooms",
16       "calendar_last_scraped",
17       "host_acceptance_rate",
18       "host_has_profile_pic",
19       "host_id",
20       "host_identity_verified",
21       "host_listings_count",
22       "host_name",
23       "host_picture_url",
24       "host_response_rate",
25       "host_response_time",
26       "host_since",
27       "host_thumbnail_url",
28       "host_total_listings_count",
29       "host_url",
30       "host_verifications",
31       "id",
32       "instant_bookable",
33       "last_scraped",
34       "latitude",
35       "listing_url",
36       "longitude",
37       "maximum_maximum_nights",
38       "maximum_minimum_nights",
```

Structure typique (quelques colonnes-clés et leur signification)

Clé	Description de la valeur
listing_url	URL publique de l’annonce
host_id, host_name	Identité de l’hôte
host_since	Date d'inscription de l’hôte
room_type	Type d’espace loué (entier, chambre privée, partagée)
property_type	Type de bien (appartement, maison, etc.)
availability_365	Nombre de jours disponibles sur l’année
number_of_reviews	Nombre total d’avis
neighbourhood_cleansed	Quartier nettoyé (format standardisé)
instant_bookable	Si le logement est réservable immédiatement
host_is_superhost	Statut de l’hôte

Quelques requêtes (1/2)

Nombre de documents : `{"$count": "nombre_de_documents"}` -> 95885



The screenshot shows the MongoDB Atlas interface for the 'airbnb' database and 'logements-paris' collection. The 'Aggregations' tab is active, displaying a pipeline with one stage: '\$count' with the field 'nombre_de_documents'. The pipeline output preview shows a single document with the value 95885. The interface includes a top navigation bar with 'Documents 96K', 'Aggregations', 'Schema', 'Indexes 1', and 'Validation'. A 'Generate' button is next to the pipeline editor. The bottom bar contains 'SAVE', 'CREATE NEW', 'EXPORT TO LANGUAGE', 'PREVIEW', 'STAGES', 'TEXT', and 'WIZARD' buttons.

localhost:27017 > airbnb > logements-paris Open MongoDB shell

Documents 96K **Aggregations** Schema Indexes 1 Validation

✦ nombre de documents Generate

▼ \$count Explain Export Run Options

Untitled - modified SAVE CREATE NEW EXPORT TO LANGUAGE PREVIEW STAGES TEXT WIZARD

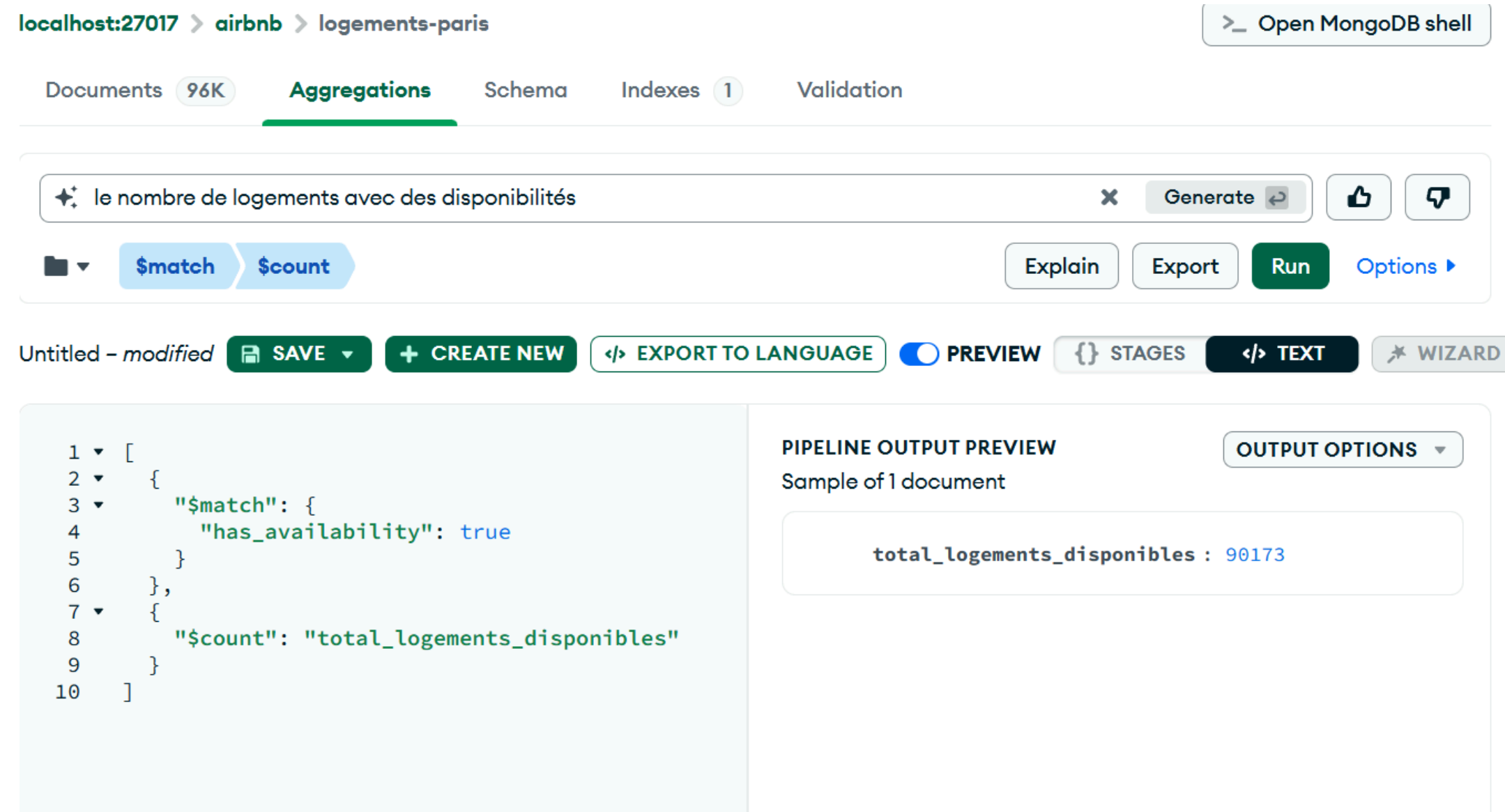
```
1 [{" $count": "nombre_de_documents" }]
```

PIPELINE OUTPUT PREVIEW OUTPUT OPTIONS
Sample of 1 document

nombre_de_documents : 95885

Quelques requêtes (2/2)

Nombre d'hôtes distincts : `{ "$match": { "has_availability": true } }, { "$count": "total_logements_disponibles" } -> 90173`



The screenshot displays the MongoDB Compass web interface. The top navigation bar shows the connection 'localhost:27017' and the database 'airbnb', with the 'logements-paris' collection selected. The 'Aggregations' tab is active, showing a pipeline with two stages: '\$match' and '\$count'. The '\$match' stage has a query: `{ "has_availability": true }`. The '\$count' stage has a field: `"total_logements_disponibles"`. The pipeline is named 'le nombre de logements avec des disponibilités'. The 'Run' button is highlighted. Below the pipeline, the 'PIPELINE OUTPUT PREVIEW' section shows a sample of 1 document with the result: `total_logements_disponibles : 90173`. The interface also includes a sidebar with 'Documents' (96K), 'Schema', 'Indexes' (1), and 'Validation' tabs. The bottom bar contains buttons for 'SAVE', 'CREATE NEW', 'EXPORT TO LANGUAGE', 'PREVIEW', 'STAGES', 'TEXT', and 'WIZARD'.

```
1 [
2   {
3     "$match": {
4       "has_availability": true
5     },
6   },
7   {
8     "$count": "total_logements_disponibles"
9   }
10 ]
```

PIPELINE OUTPUT PREVIEW
Sample of 1 document

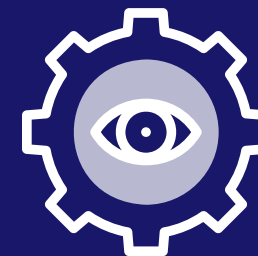
```
total_logements_disponibles : 90173
```

Pérenniser l'utilisation et prévenir les incidents

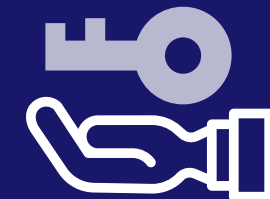
Mesures concrètes pour garantir disponibilité,
intégrité et sécurité des données



Mise en place de
sauvegardes régulières et
procédures de restauration
testées pour garantir la
disponibilité des données.



Surveillance et alerting pour
détecter les anomalies et
prévenir les incidents.



Stratégies de gouvernance et
de contrôle d'accès pour
préserver l'intégrité et la
confidentialité des données.



Sauvegardes Régulières & Procédures De Restauration

Ne Plus Jamais Perdre De Données, Même Après Un Crash Disque, Corruption De Fichier, Ou Suppression Accidentelle.

- Eviter Backup CSV/JSON dans Compass (manuel.. & les dates, nombres, ObjectIds deviennent des chaînes)
- Sauvegarde automatique via mongodump + cron (en local)
- Bonus : Sauvegardes dans le cloud avec MongoDB Atlas (disponibles selon el plan, restauration point-in-time, Interface web)



Surveillance Et Alerting Des Incidents

Être prévenu si MongoDB plante, dépasse les limites, ou si la base devient instable.

- Surveillance locale avec mongostat et mongotop
- MongoDB Atlas Monitoring même en local
- Integrations avec Prometheus + Grafana (Dashboard Grafana custom (CPU, RAM, requêtes lentes, etc.) + Alertes : email, Slack, Telegram...



Créer Des Utilisateurs Avec Rôles Spécifiques

Protéger l'intégrité et la confidentialité des données en limitant les droits.

- Activer la sécurité dans mongod.conf
- Créer un admin, un user lecteur, un analyste read write
- Chiffrement local (via filesystem ou "Encrypted Storage Engine" dans la version Enterprise)

Requêtes analytiques principales

Questions clés pour explorer les annonces et les hôtes

■ Nombre d'annonces par type de location.

Compte des annonces classées par catégorie de logement.

■ Liste des logements les plus loués

basés sur le nombre de reviews par mois.

■ Nombre total d'hôtes différents.

Nombre distinct d'hôtes présents sur la plateforme.

■ Nombre de locations réservables instantanément.

Compte des annonces avec réservation instantanée activée.

■ Liste des hôtes ayant plus de 100 annonces sur la plateforme.

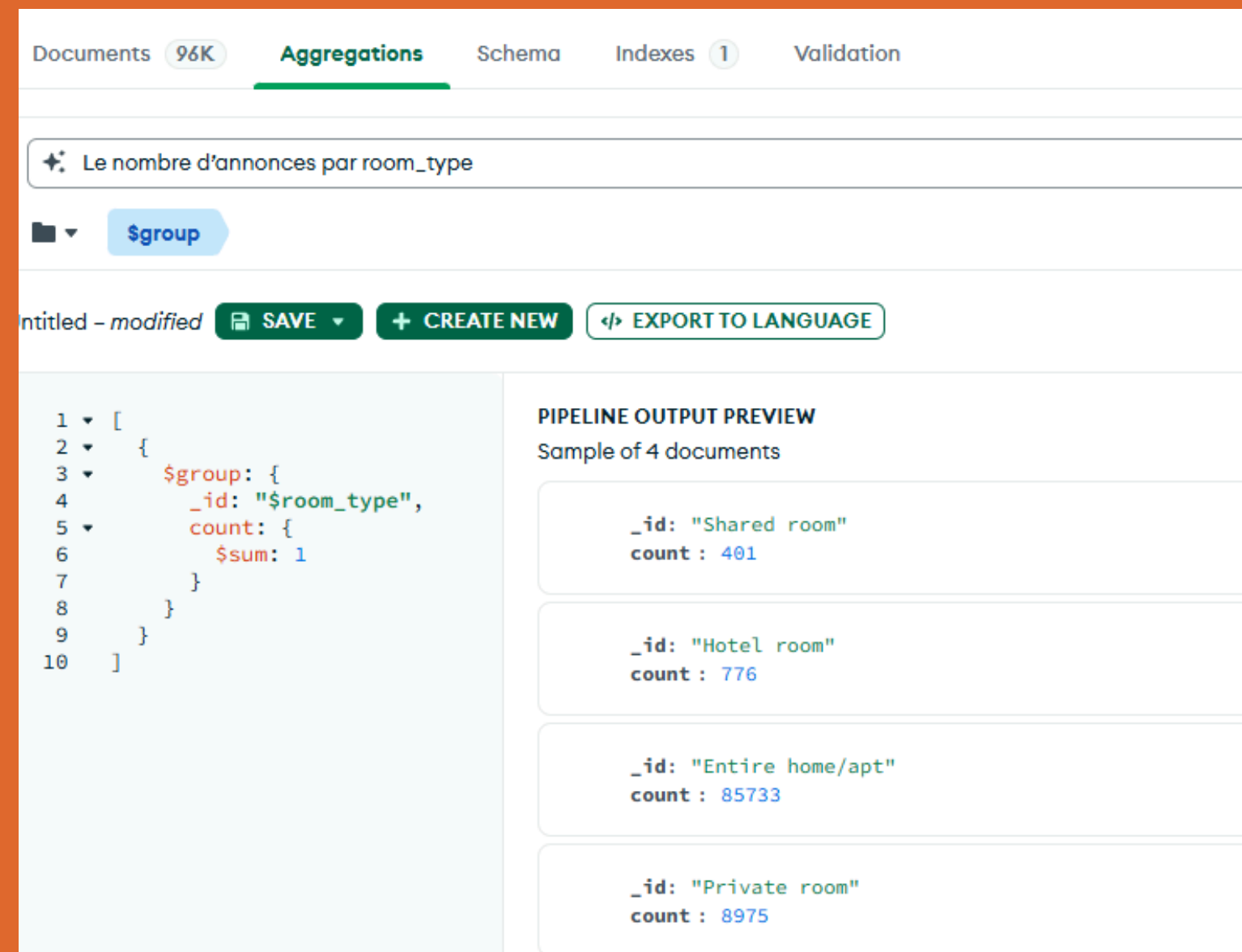
Avec leur noms , leur localisation et leur total_listing

■ Nombre de super hôtes différents.

Compte des hôtes labellisés « super hôte ».

requêtes analytiques (1/6)

Nombre d'annonces par type de location. : \$group: _id: "\$room_type", count: \$sum: 1

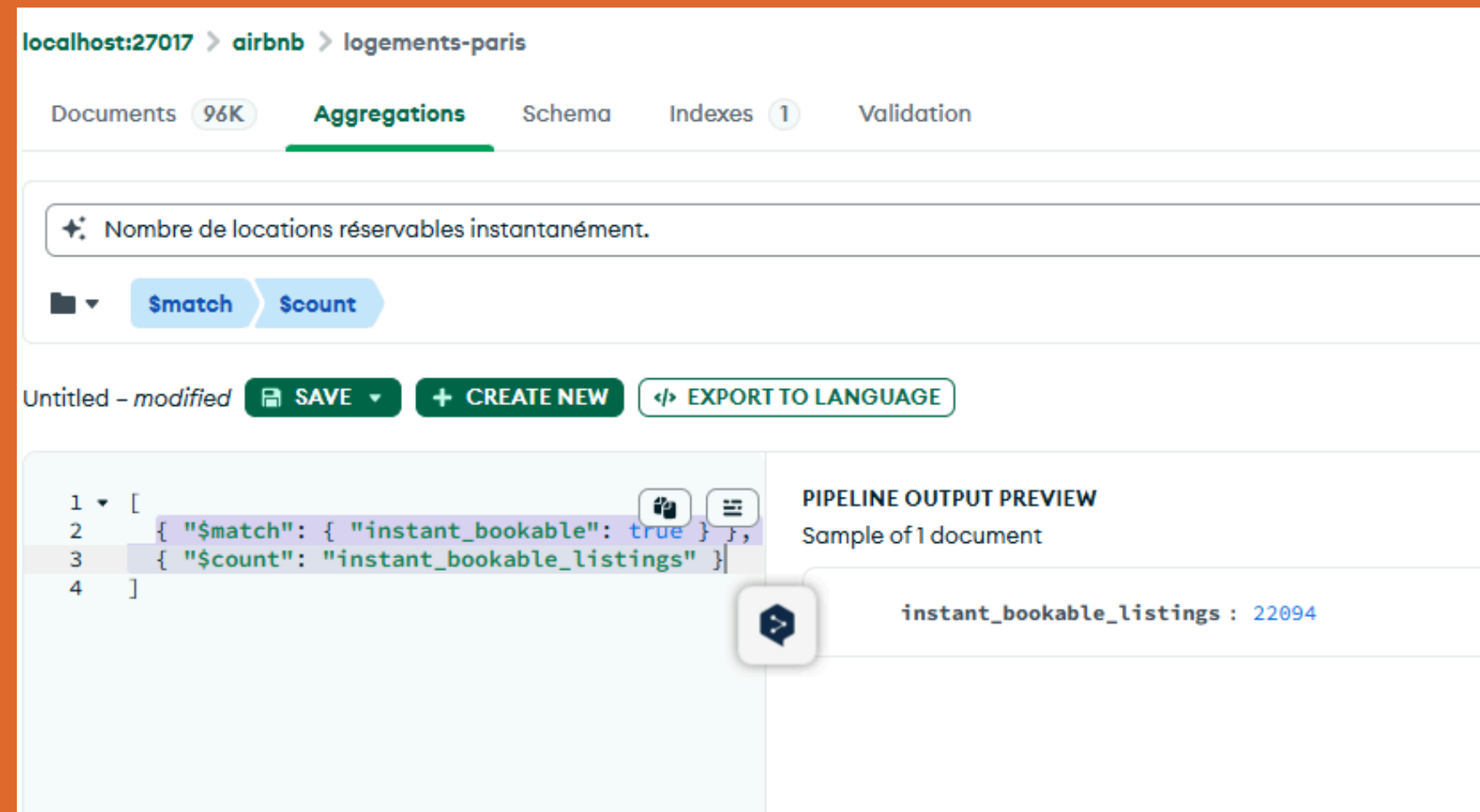


The screenshot shows the MongoDB Aggregations interface. The title bar indicates 96K documents, 1 index, and the Aggregations tab is active. The pipeline title is "Le nombre d'annonces par room_type". A dropdown menu shows the aggregation stage as "\$group". Below the title bar, there are buttons for "SAVE", "CREATE NEW", and "EXPORT TO LANGUAGE". The main area is split into two panels: a code editor on the left and a "PIPELINE OUTPUT PREVIEW" on the right. The code editor shows a JSON array with one document containing a \$group stage. The preview panel shows a sample of 4 documents with the following data:

_id	count
"Shared room"	401
"Hotel room"	776
"Entire home/apt"	85733
"Private room"	8975

requêtes analytiques (2/6)

Nombre de locations réservables instantanément : { "\$match": { "instant_bookable": true } } { "\$count":
"instant_bookable_listings" }
22094



The screenshot shows the MongoDB Compass interface for a database named 'airbnb' and a collection named 'logements-paris'. The 'Aggregations' tab is active, displaying a query pipeline with two stages: '\$match' and '\$count'. The query text is: { "\$match": { "instant_bookable": true } }, { "\$count": "instant_bookable_listings" }. The result of the aggregation is shown in the 'PIPELINE OUTPUT PREVIEW' section, indicating a count of 22094 for the 'instant_bookable_listings' field.

localhost:27017 > airbnb > logements-paris

Documents 96K Aggregations Schema Indexes 1 Validation

✦ Nombre de locations réservables instantanément.

\$match \$count

Untitled - modified SAVE + CREATE NEW </> EXPORT TO LANGUAGE

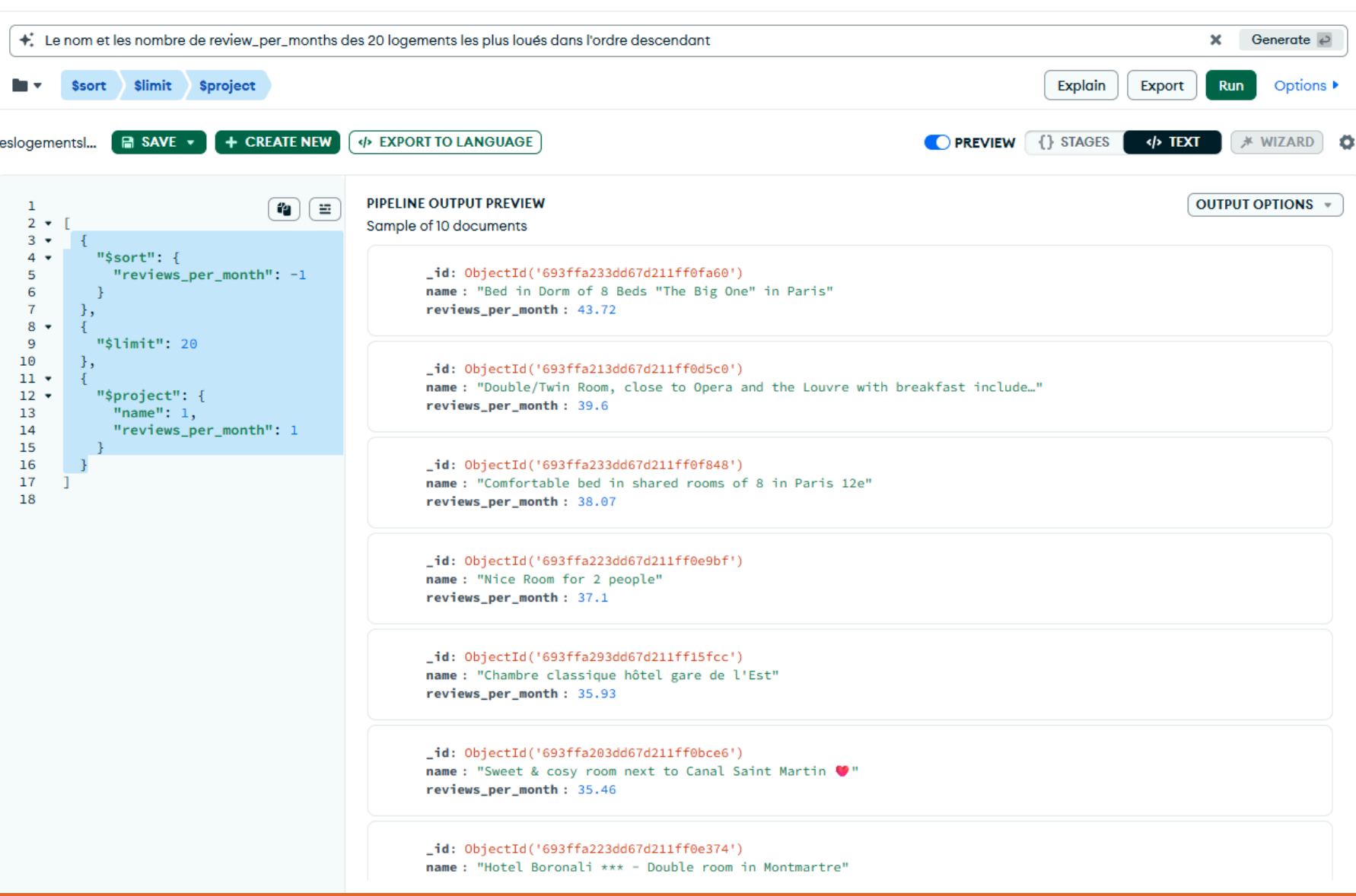
```
1 [
2   { "$match": { "instant_bookable": true } },
3   { "$count": "instant_bookable_listings" }
4 ]
```

PIPELINE OUTPUT PREVIEW
Sample of 1 document

instant_bookable_listings : 22094

requêtes analytiques (3/6)

Liste des logements les plus loués : `"$sort": { "reviews_per_month": -1 } }, { "$limit": 20 }, { "$project": { "name": 1, "reviews_per_month": 1 } }`



The screenshot shows the MongoDB Atlas query editor interface. The query is written in the left pane and the results are shown in the right pane.

Query:

```
1  [
2  {
3    "$sort": {
4      "reviews_per_month": -1
5    },
6  },
7  {
8    "$limit": 20
9  },
10 },
11 {
12   "$project": {
13     "name": 1,
14     "reviews_per_month": 1
15   }
16 }
17 ]
18
```

PIPELINE OUTPUT PREVIEW
Sample of 10 documents

<code>_id: ObjectId('693ffa233dd67d211ff0fa60')</code> <code>name: "Bed in Dorm of 8 Beds "The Big One" in Paris"</code> <code>reviews_per_month: 43.72</code>
<code>_id: ObjectId('693ffa213dd67d211ff0d5c0')</code> <code>name: "Double/Twin Room, close to Opera and the Louvre with breakfast include..."</code> <code>reviews_per_month: 39.6</code>
<code>_id: ObjectId('693ffa233dd67d211ff0f848')</code> <code>name: "Comfortable bed in shared rooms of 8 in Paris 12e"</code> <code>reviews_per_month: 38.07</code>
<code>_id: ObjectId('693ffa223dd67d211ff0e9bf')</code> <code>name: "Nice Room for 2 people"</code> <code>reviews_per_month: 37.1</code>
<code>_id: ObjectId('693ffa293dd67d211ff15fcc')</code> <code>name: "Chambre classique hôtel gare de l'Est"</code> <code>reviews_per_month: 35.93</code>
<code>_id: ObjectId('693ffa203dd67d211ff0bce6')</code> <code>name: "Sweet & cosy room next to Canal Saint Martin ❤️"</code> <code>reviews_per_month: 35.46</code>
<code>_id: ObjectId('693ffa223dd67d211ff0e374')</code> <code>name: "Hotel Boronali *** - Double room in Montmartre"</code>

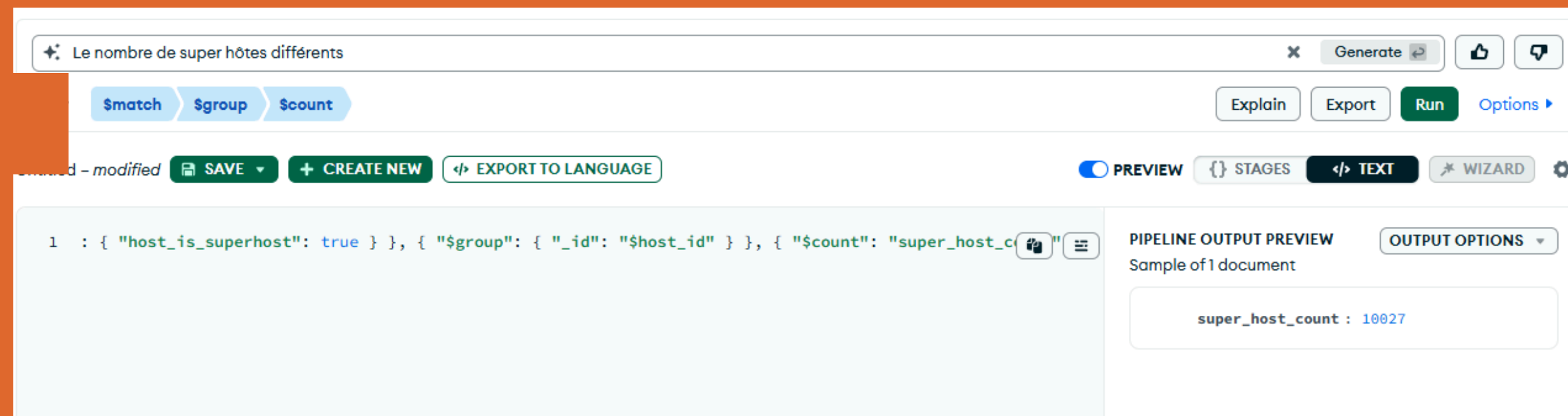
requêtes analytiques / (4/6)

Le nombre total d'hôtes différents: { "\$group": { "_id": "\$host_id" } }, { "\$count": "total_different_hosts" } } ->
71979

The screenshot shows a web-based interface for running data queries. At the top, a text input field contains the query: "nombre total d'hôtes différents". Below this, a pipeline editor shows a single stage with the aggregation pipeline: `{ "$group": { "_id": "$host_id" } }, { "$count": "total_different_hosts" } }`. The interface includes buttons for "Generate", "Explain", "Export", "Run", and "Options". Below the pipeline editor, there are buttons for "modified", "SAVE", "CREATE NEW", and "EXPORT TO LANGUAGE". On the right side, there are toggle switches for "PREVIEW", "STAGES", "TEXT", and "WIZARD". The "PIPELINE OUTPUT PREVIEW" section shows a sample of 1 document with the output: `total_different_hosts : 71979`. The "OUTPUT OPTIONS" dropdown is also visible.

requêtes analytiques / (5/6)

Le nombre total de super-hôtes différents: `{ "$match": { "host_is_superuser": true } }, { "$group": { "_id": "$host_id" } }, { "$count": "super_host_count" } -> 10027`



The screenshot displays the MongoDB Atlas query builder interface. The title bar shows the query name "Le nombre de super hôtes différents". The pipeline stages are defined as follows:

- `$match`: `{ "host_is_superuser": true }`
- `$group`: `{ "_id": "$host_id" }`
- `$count`: `"super_host_count"`

The interface includes buttons for "Generate", "Explain", "Export", "Run", and "Options". Below the pipeline editor, there are buttons for "SAVE", "CREATE NEW", and "EXPORT TO LANGUAGE". The "PIPELINE OUTPUT PREVIEW" section shows a sample of 1 document with the result:

```
super_host_count : 10027
```

requêtes analytiques / (6/6)

Les hôtes avec plus de 100 annonces

Le nom, la date de creation et la location des hôtes aet leur nombre d'annonces avec plus de 100 annonces par nombre descendant

Generate

\$group \$match \$sort \$project

Explain Export Run Options

Untitled - modified SAVE + CREATE NEW EXPORT TO LANGUAGE

PREVIEW STAGES TEXT WIZARD

```
1  [
2  {
3    "$group": {
4      "_id": "$host_id",
5      "host_name": {"$first": "$host_name"},
6      "host_since": {"$first": "$host_since"},
7      "host_location": {"$first": "$host_location"},
8      "total_listings": {"$first": "$host_total_listings_count"}
9    },
10   {"$match": {"total_listings": {"$gt": 100}}},
11   {"$sort": {"total_listings": -1}},
12   {"$project": {
13     "host_name": 1,
14     "host_since": 1,
15     "host_location": 1,
16     "total_listings": 1
17   }}
18 ]
19
```

PIPELINE OUTPUT PREVIEW

Sample of 10 documents

OUTPUT OPTIONS

_id: 439074505
host_name : "Travelnest"
host_since : 2022-01-05T00:00:00.000+00:00
host_location : null
total_listings : 6278

_id: 337698666
host_name : "Agence COCOONR / BOOK&PAY"
host_since : 2020-02-20T00:00:00.000+00:00
host_location : "Rennes, France"
total_listings : 3256

_id: 270230184
host_name : "Janna - BELVILLA"
host_since : 2019-06-21T00:00:00.000+00:00
host_location : "Zürich, Switzerland"
total_listings : 1954

_id: 400123061
host_name : "Maeva"
host_since : 2021-05-05T00:00:00.000+00:00
host_location : null
total_listings : 1665

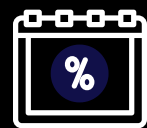
_id: 402191311
host_name : "GuestReady"
host_since : 2021-05-18T00:00:00.000+00:00
host_location : null
total_listings : 1631

Analyse des statistiques et métriques avancées

Indicateurs clés de performance pour les logements et quartiers

01

Taux de réservation moyen par mois par type de logement.



Moyenne mensuelle des réservations selon le type de logement.

02

Médiane du nombre d'avis pour tous les logements comme proxy du nombre de réservations.



Utilisation de la médiane des avis comme indicateur de réservation.

03

Médiane des nombres d'avis par catégorie d'hôte (super hôte vs non super hôte).



Comparaison des médianes d'avis entre catégories d'hôte.

04

Densité de logements par quartier de Paris.



Cartographie de la répartition des logements par quartier.

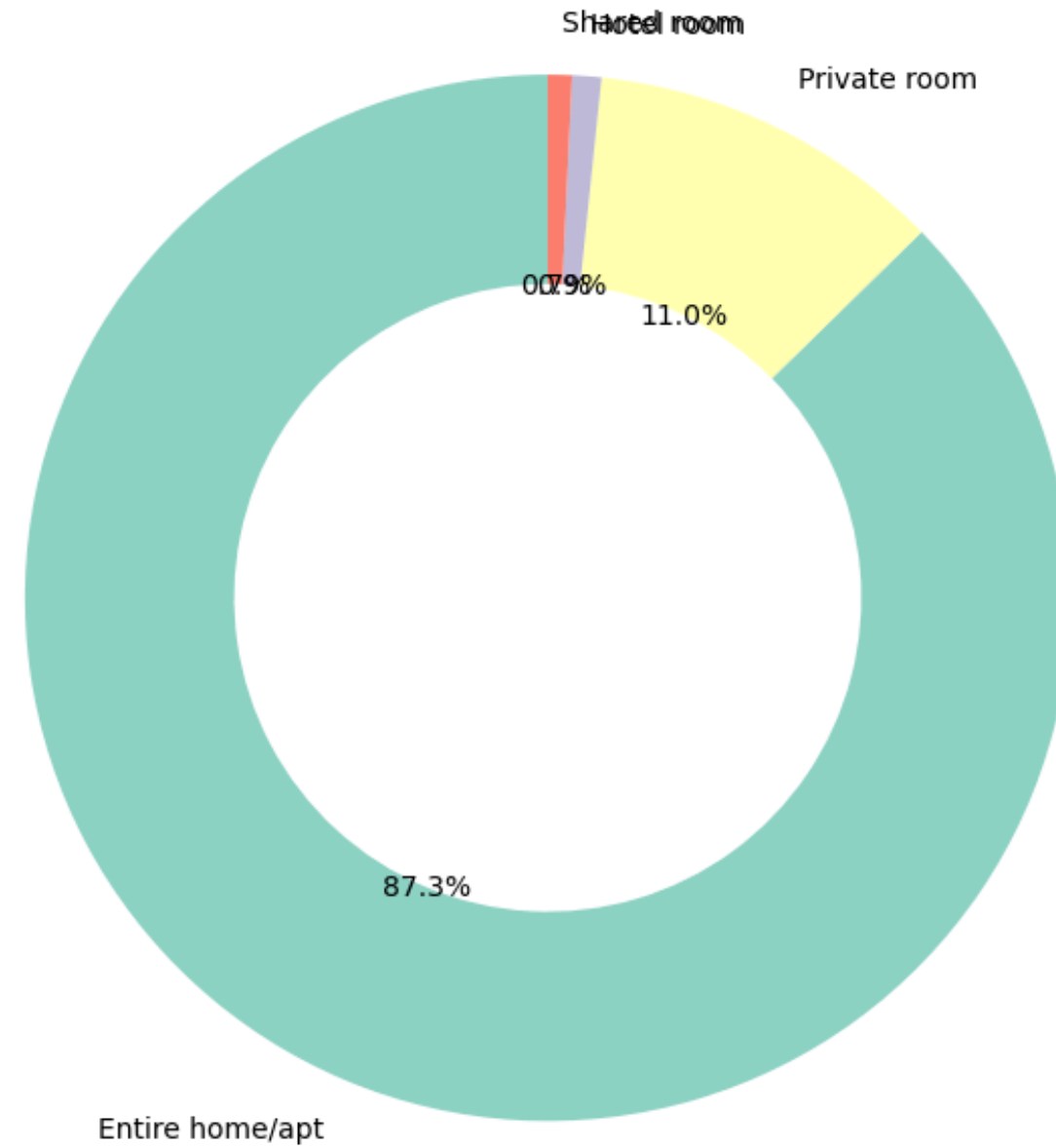
05

Quartiers avec le plus fort taux de réservation par mois.



Identification des quartiers les plus performants en réservations.

	room_type	total_reviews	proportion
0	Entire home/apt	62380.39	87.33
1	Private room	7886.82	11.04
2	Hotel room	639.44	0.90
3	Shared room	526.03	0.74

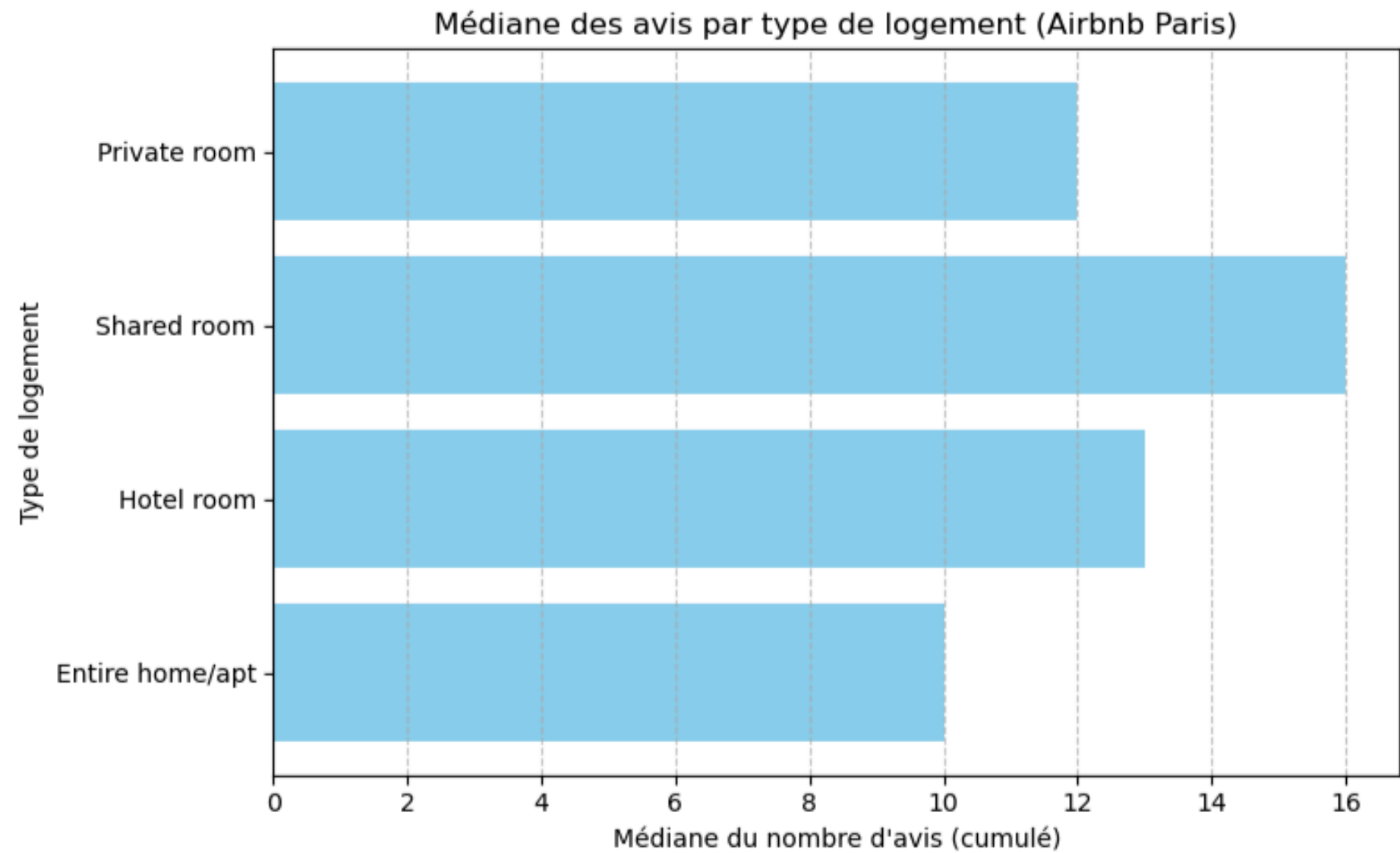


statistiques (1/6)

Le taux de réservation moyen par mois par type de logement

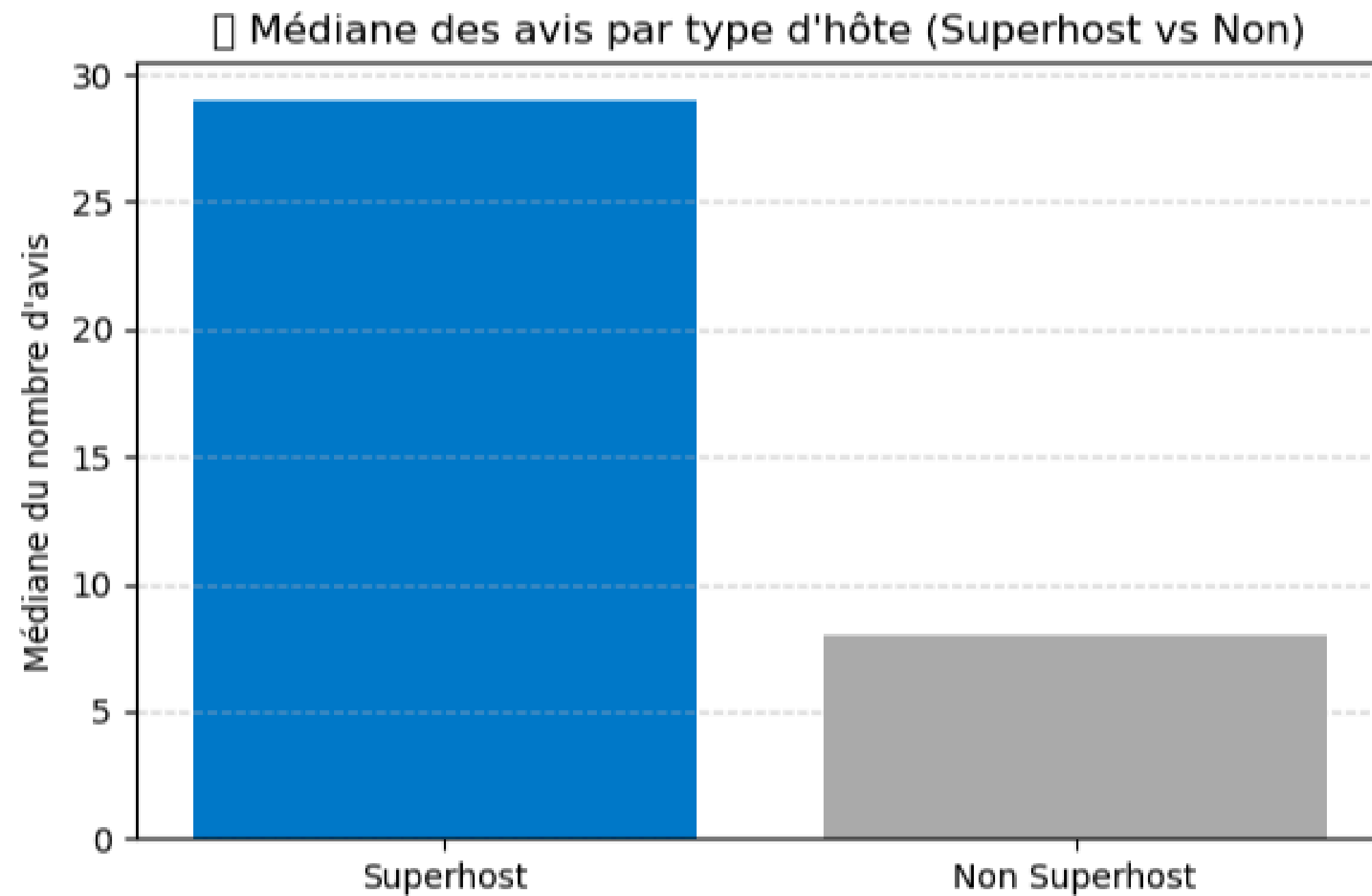
✓ Aperçu des données :

	room_type	median_review_count
0	Entire home/apt	10.0
1	Hotel room	13.0
2	Shared room	16.0
3	Private room	12.0



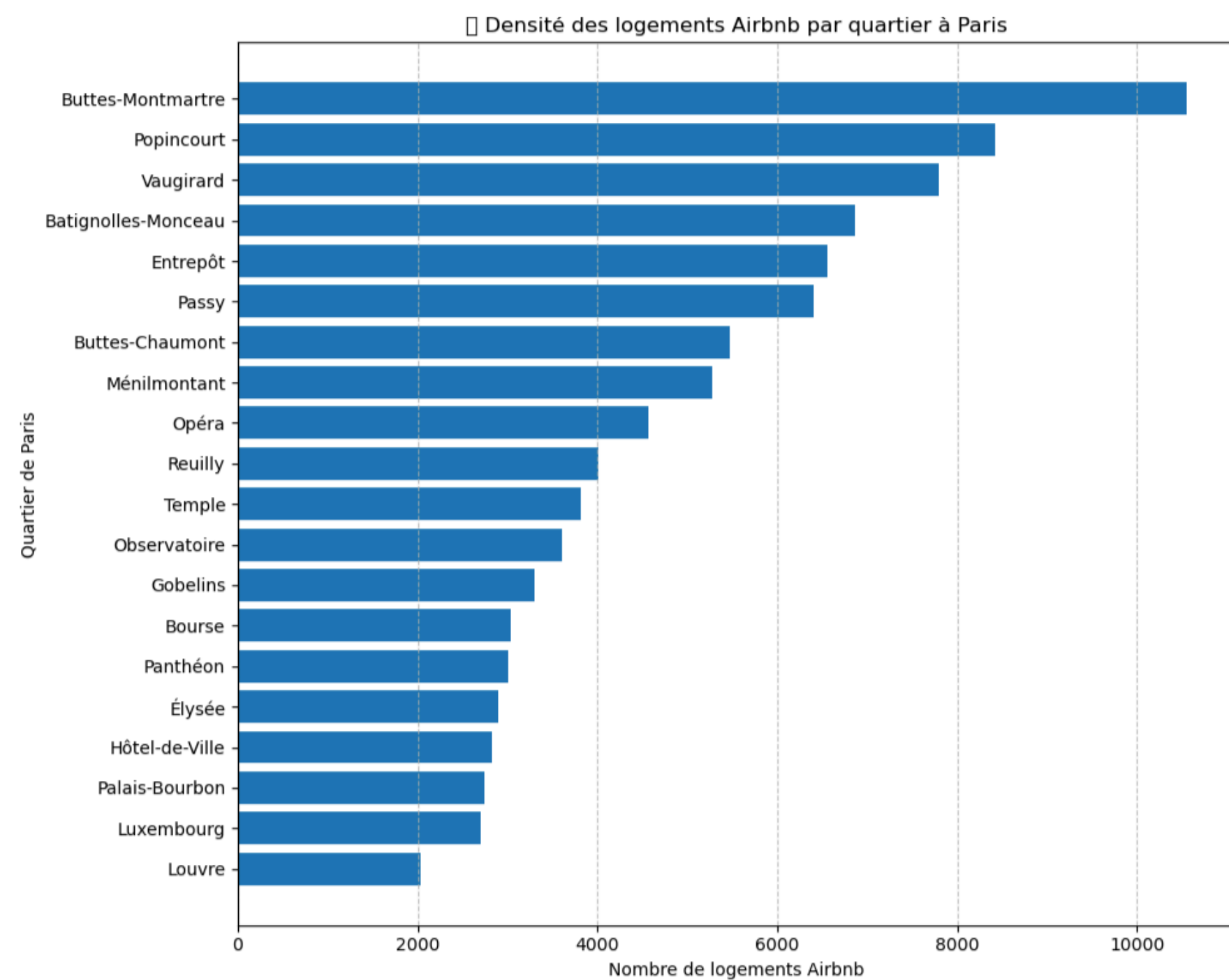
statistiques (2/6)

Médiane des avis par type de logement (Airbnb Paris)



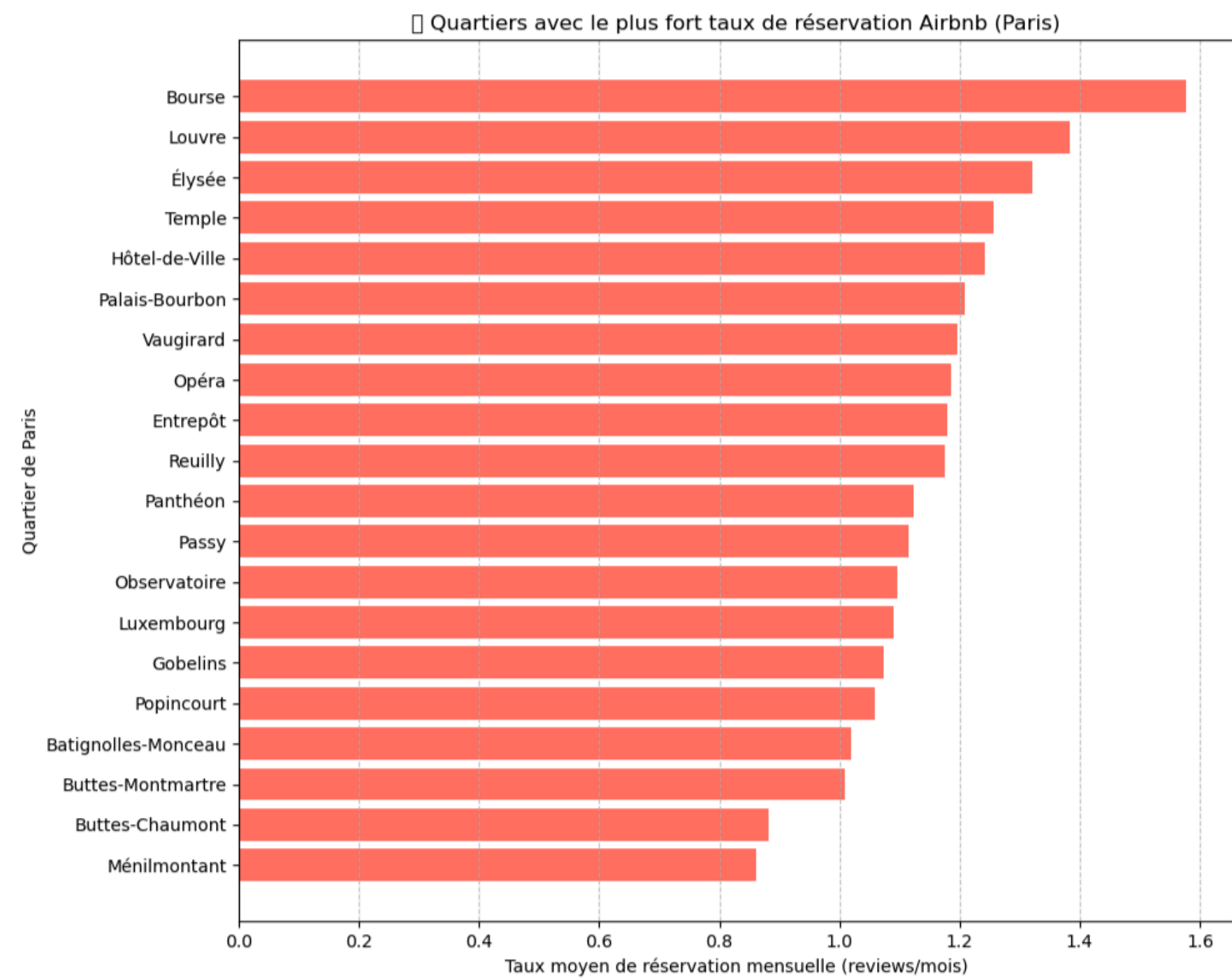
statistiques (3/6)

Médiane des avis par type d'hôte (Superhost vs Non)



statistiques (4/6)

La densité de logements par quartier de Paris



statistiques (5/6)

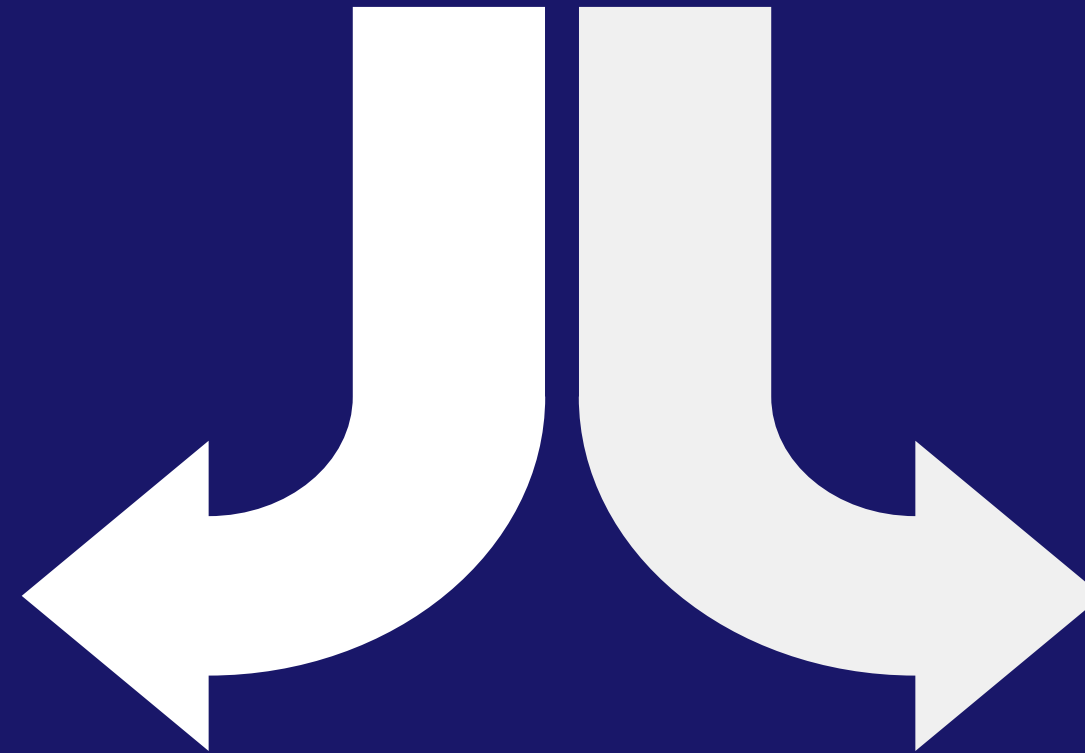
Les quartiers avec le plus fort taux de réservation par mois

Visualisations BI et alimentation des données

Captures d'écran et processus de transfert depuis MongoDB

Captures d'écran du tableau de bord

Captures d'écran du tableau de bord BI montrant les visualisations principales.



Description du processus d'alimentation

Description du processus d'alimentation depuis MongoDB vers l'outil BI avec captures d'écran étape par étape.

Schéma du SI au complet

combinaison MingoDB et PowerBi

MongoDB Atlas (cloud) → API REST sécurisée (Node.js/Express ou MongoDB Data API)

|

| → Sauvegarde automatique (mongodump + cron job)

|

▼

ETL/ELT (via Azure Data Factory ou Python + Scheduled Job)

▼

Azure Blob Storage (intermédiaire optionnel) ou Dataset CSV/JSON

▼

Power BI (via connecteur Web/API ou fichiers importés)

↓

Architecture

- MongoDB Atlas (ou instance en ligne)
- API (accès temps réel aux données sans exposer la base.) via connecteur Web (GET)

Architecture de collecte et connexion MongoDB to Power BI

Processus, rôles, sécurité et avantages clés

Avantages de cette architecture

- Fidélité : les données viennent directement de la source via API.
- Actualité : actualisation fréquente configurée automatiquement.
- Disponibilité : architecture cloud et découplée, avec redondance (sauvegarde, API, fichier intermédiaire)

Sécurité et disponibilité

- Chiffrement des connexions (SSL/TLS) entre MongoDB/API/Power BI.
- Token ou OAuth pour authentification API.
- Monitoring avec alerting sur échec de l'ETL ou de l'actualisation Power BI.
- Accès API journalisé.

Gestion des rôles utilisateurs

- Admin: `readWriteAnyDatabase` sur MongoDB; Création/suppression d'API; Administration + gestion sources sur Power BI.
- Lecteur: `read` sur MongoDB; Lecture via API (read-only); Accès aux rapports en lecture sur Power BI.

MongoDB Atlas (ou instance en ligne)

- La base `airbnb` contient la collection `logements-airbnb`.
- Mise en place d'un plan de sauvegarde automatique avec `mongodump` et `cron`.
- Ajout d'un **audit trail** pour détecter les modifications (utile pour l'incrémental).

API REST ou MongoDB Data API

- Expose des endpoints REST (`GET /logements`) filtrés et paginés.
- Authentification via API Key ou OAuth2.
- Permet l'accès temps réel aux données sans exposer la base.

Pipeline d'extraction (ETL ou ELT)

- Outils recommandés : Azure Data Factory, Python script via Azure Function, ou Power Automate.
- Mode d'actualisation : toutes les heures ou selon un déclencheur.
- Transformation possible en JSON/CSV standardisé pour Power BI.

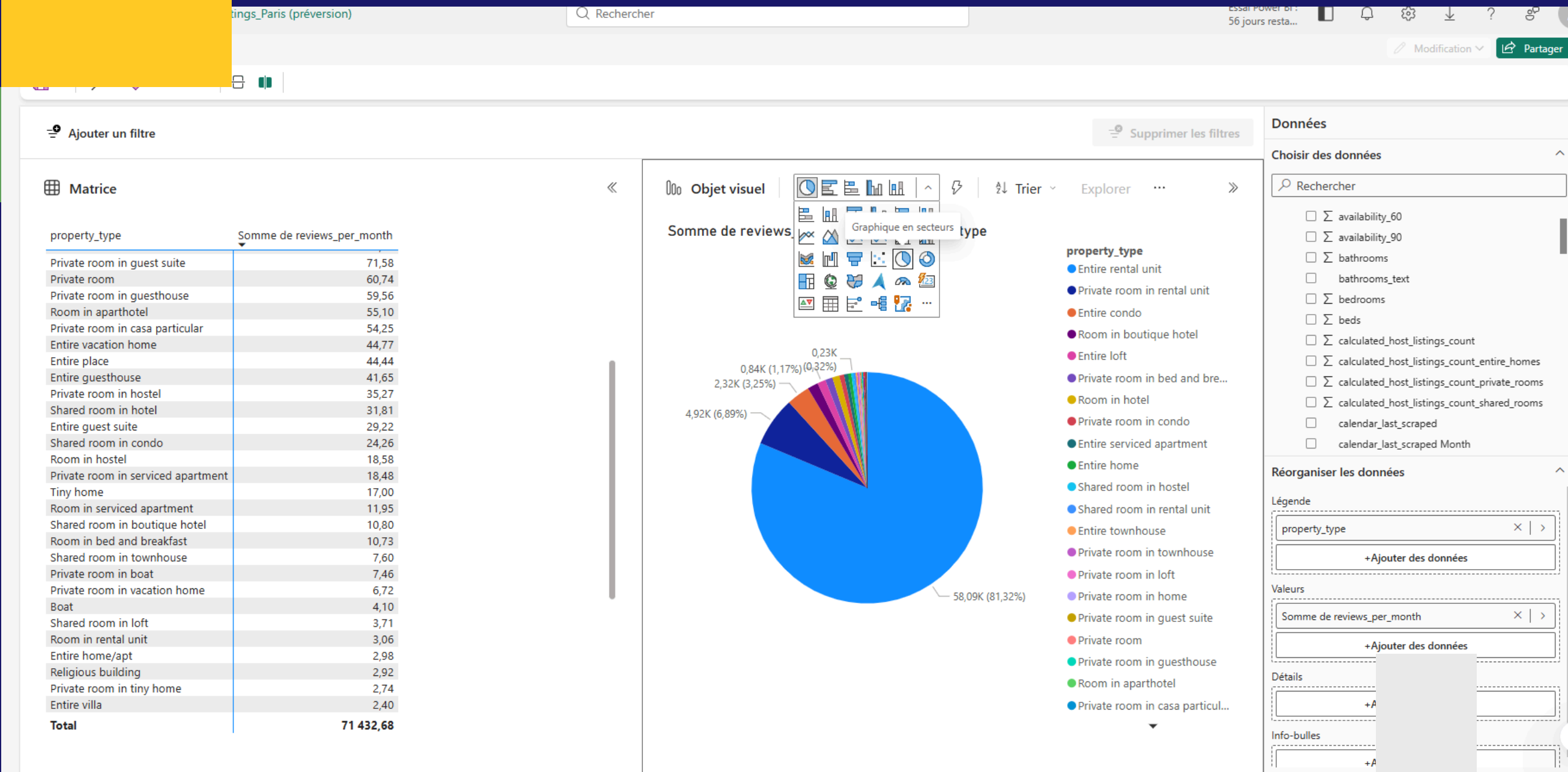
Connexion Power BI

- Power BI utilise l'API via **connecteur Web (GET)** ou **Azure Blob/OneDrive** pour les fichiers plats.
- Configuration de l'actualisation automatique (Pro/Premium nécessaire pour +8 refreshes/jour).



Power BI

Un exemple de visualisation



My Workspace

> Explorer listing_Paris

```
  _id: 'Entire home/apt',  
  total_annonces: 85733  
}  
{  
  _id: 'Private room',  
  total_annonces: 8975  
}  
{  
  _id: 'Hotel room',  
  total_annonces: 776  
}  
{  
  _id: 'Shared room',  
  total_annonces: 401  
}
```

Combien D'annonces Y A-T-Il Par Type De Location ?

Requête CLI Mongosh (1/6)

```
db"logements-paris".aggregate( { $group: { _id: "$room_type", total_annonces: { $sum: 1 } } }, { $sort: { total_annonces: -1 } }
```

```
< {
  name: 'Sweet & cosy room next to Canal Saint Martin ❤️',
  number_of_reviews: 3067
}
{
  name: 'Double/Twin Room, close to Opera and the Louvre with breakfast included',
  number_of_reviews: 2620
}
{
  name: 'Bed in Dorm of 8 Beds "The Big One" in Paris',
  number_of_reviews: 2294
}
{
  name: 'Comfortable bed in shared rooms of 8 in Paris 12e',
  number_of_reviews: 2105
}
{
  name: 'Nice Room for 2 people',
  number_of_reviews: 2048
}
```

Quelles Sont Les 5 Annonces De Location Avec Le Plus D'évaluations ? Et Combien D'évaluations Ont-Elles ?

Requête CLI Mongosh (2/6)

```
db"logements-paris".find( {}, { name: 1, number_of_reviews: 1, _id: 0 } ) sort({ number_of_reviews: -1 }).limit(5)
```

Quel Est Le Nombre Total D'hôtes Différents ?

Requête CLI Mongosh (3/6)

`db"logements-paris".distinct("host_id").length`

71979

```
>_MONGOSH
```

```
>use db["logements-paris"].distinct("host_id").length
```

```
< 71979
```

```
airbnb> |
```

```
< {  
  total_annonces: 95885,  
  instant_bookables: 22094,  
  proportion: 23.04  
}
```

Quel Est Le Nombre De Locations Réservables Instantanément ? Quelle Proportion ?

Requête CLI Mongosh (4/6)

```
db"logements-paris".aggregate( { $group: { _id: "$instant_bookable", count: { $sum: 1 } } }, {  
  $group: { _id: null, total: { $sum: "$count" }, instant_bookable_count: { $sum: { $cond: { $eq:  
    "$_id", true }, "$count", 0 } } } }, { $project: { _id: 0, total_annonces: "$total", instant_bookables:  
    "$instant_bookable_count", proportion: { $round: { $multiply: { $divide:  
      "$instant_bookable_count", "$total" }, 100 }, 2 } } } } )
```

```
total_hotes: 71979,  
total_100plus: 24,  
liste_hotes: [  
  {  
    total_annonces: 132,  
    host_id: 440295601,  
    host_name: 'Parisian Home'  
  },  
  {  
    total_annonces: 170,  
    host_id: 125797498,  
    host_name: 'Welkeys'  
  },  
  {  
    total_annonces: 730,  
    host_id: 314994947,  
    host_name: 'Blueground'  
  },  
  {  
    total_annonces: 426,  
    host_id: 50502817,  
    host_name: 'Pierre De WeHost'  
  },  
]
```

Est-Ce Des Hôtes Ont Plus De 100 Annonces Sur Les Plateformes ?

Requête CLI Mongosh (5/6)

Cela représente quel pourcentage des hôtes ?

Réponse : 3%

Combien Y A-T-Il De Super Hôtes Différents ?

Requête CLI Mongosh (5/6)

```
< {  
  total_hotes: 71979,  
  super_hotes: 10027,  
  proportion: 13.93  
}
```

Requêtes complexes avec Polars



Objectif

transmettre les résultats à l'équipe parisienne de l'association.

Requêtes à exécuter

1. Calculer le taux de réservation moyen par mois par type de logement
2. Calculer la médiane des nombre d'avis pour tous les logements
3. Calculer la médiane des nombre d'avis par catégorie d'hôte
4. Calculer la densité de logements par quartier de Paris
5. Identifier les quartiers avec le plus fort taux de réservation par mois

```
        "reviews_per_month": 1
    }
)

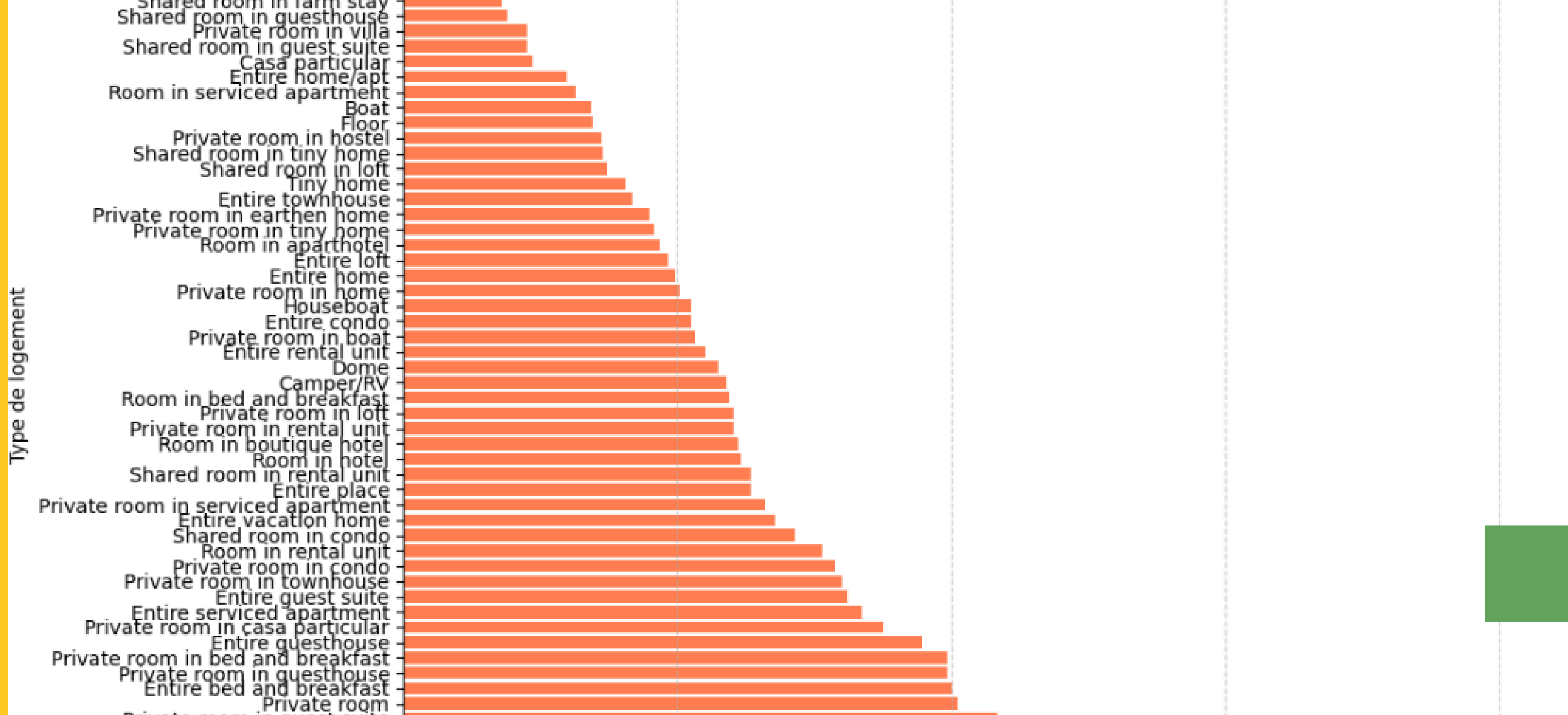
rows = list(cursor)
if not rows:
    raise ValueError("❌ Aucune donnée valide trouvée.")

# 3 Chargement dans Polars
df = pl.DataFrame(rows)

# 4 Agrégation (API Polars correcte)
agg = (
    df
    .group_by("property_type") # ✅ group_by et non groupby
    .agg([
```

Requête ++ (1/5)

Taux de réservation moyen par mois par type de logement



Requête ++ (1/5)

Taux de réservation moyen par mois par type de logement

```

# 2 Extraction des données : number_of_reviews
cursor = collection.find(
    {
        "number_of_reviews": {"$exists": True, "$ne": None}
    },
    {
        "_id": 0,
        "number_of_reviews": 1
    }
)

rows = list(cursor)
if not rows:
    raise ValueError("❌ Aucune donnée avec 'number_of_reviews' trouvée.")

# 3 Chargement dans Polars
df = pl.DataFrame(rows)

# 4 Médiane
median_reviews = df.select(pl.median("number_of_reviews")).item()
print(f"📊 Médiane du nombre d'avis : {median_reviews}")

# 5 Boxplot avec limitation de l'axe X
plt.figure(figsize=(10, 2.5)) # petite hauteur pour visuel "compact"
plt.boxplot(df["number_of_reviews"], vert=False,
            patch_artist=True,
            boxprops=dict(facecolor="lightblue", color="blue"),
            medianprops=dict(color="red", linewidth=2),
            whiskerprops=dict(color="gray"),
            capprops=dict(color="gray"),
            flierprops=dict(marker=".", markersize=3, color="gray", linestyle="none"))

plt.title("Distribution du nombre d'avis par logement (Airbnb Paris)")
plt.xlabel("Nombre d'avis")
plt.xlim(0, 200) # ✅ Limite volontaire pour améliorer la lisibilité
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

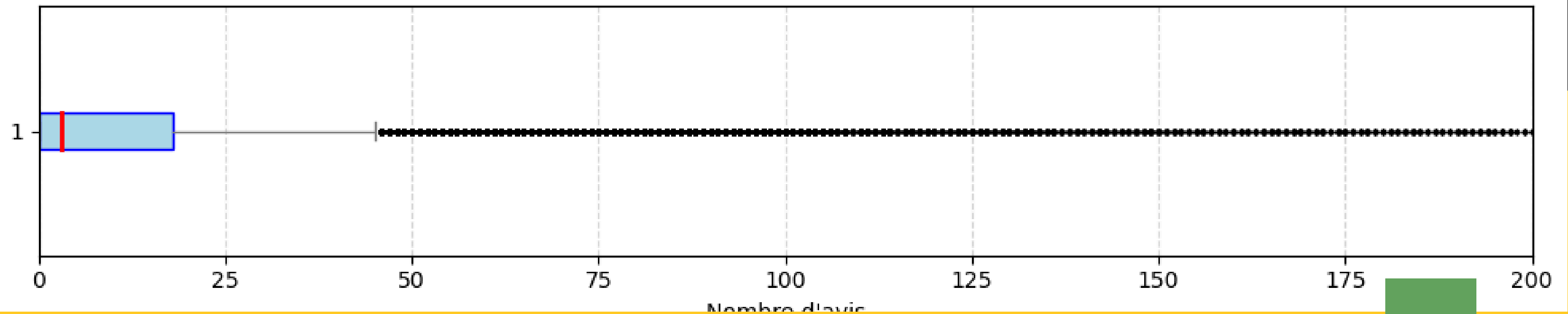
```

Requête ++ (2/5)

Calculer la médiane des nombre d'avis pour tous les logements

Médiane du nombre d'avis : 3.0

Distribution du nombre d'avis pour tous les logements (Airbnb Paris)



Requête ++ (2/5)

Calculer la médiane des nombre d'avis pour tous les logements

```
        "host_is_superhost": 1,
        "number_of_reviews": 1
    }
)

rows = list(cursor)
if not rows:
    raise ValueError("❌ Aucune donnée trouvée avec 'host_is_superhost' et 'number_of_reviews'.")

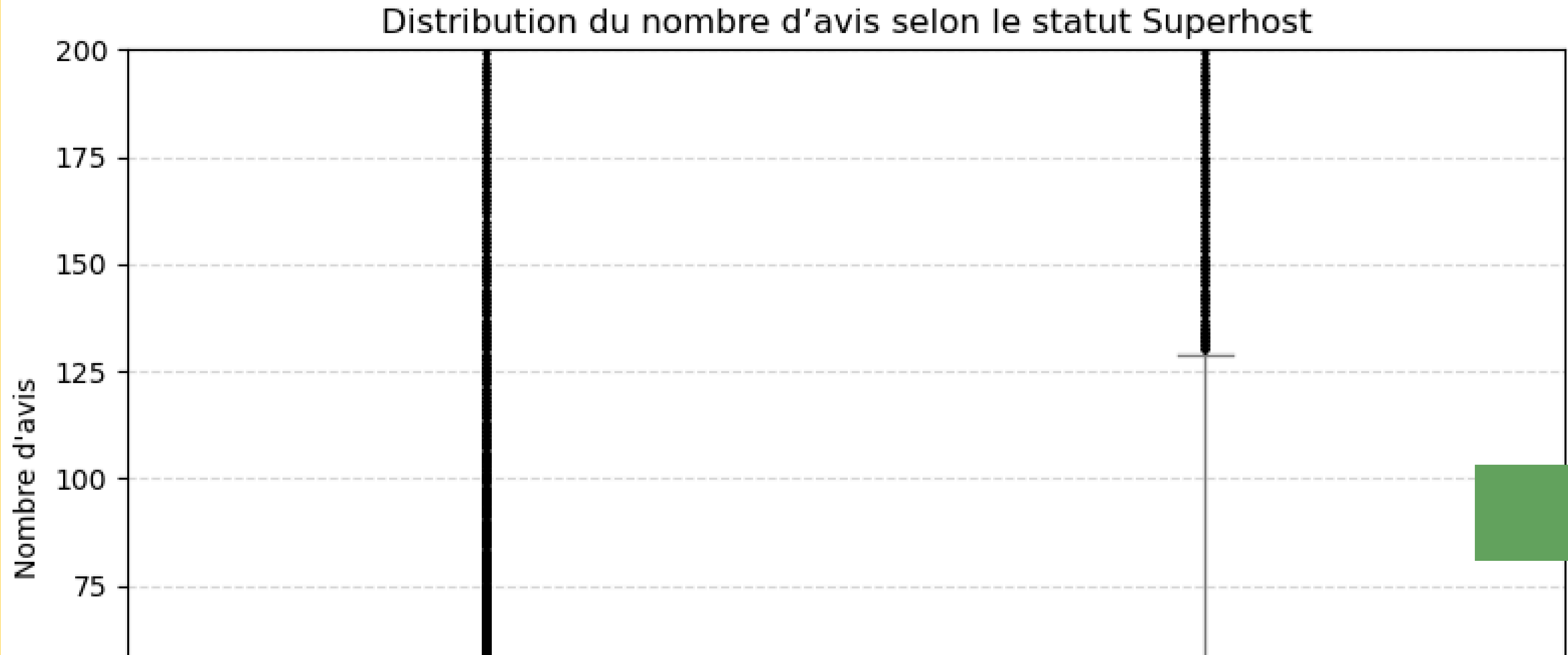
# 3 Chargement dans Polars
df = pl.DataFrame(rows)

# 4 Calcul des médianes par groupe
grouped = (
    df
    .group_by("host_is_superhost")
    .agg([
        pl.median("number_of_reviews").alias("median_reviews")
    ])
)
```

Requête ++ (3/5)

Calculer la médiane des nombre d'avis par catégorie d'hôte

True	24.6	14776
------	------	-------



Requête ++ (3/5)

Calculer la médiane des nombre d'avis par catégorie d'hôte

```
},
{
    "_id": 0,
    "neighbourhood_cleansed": 1
}
)

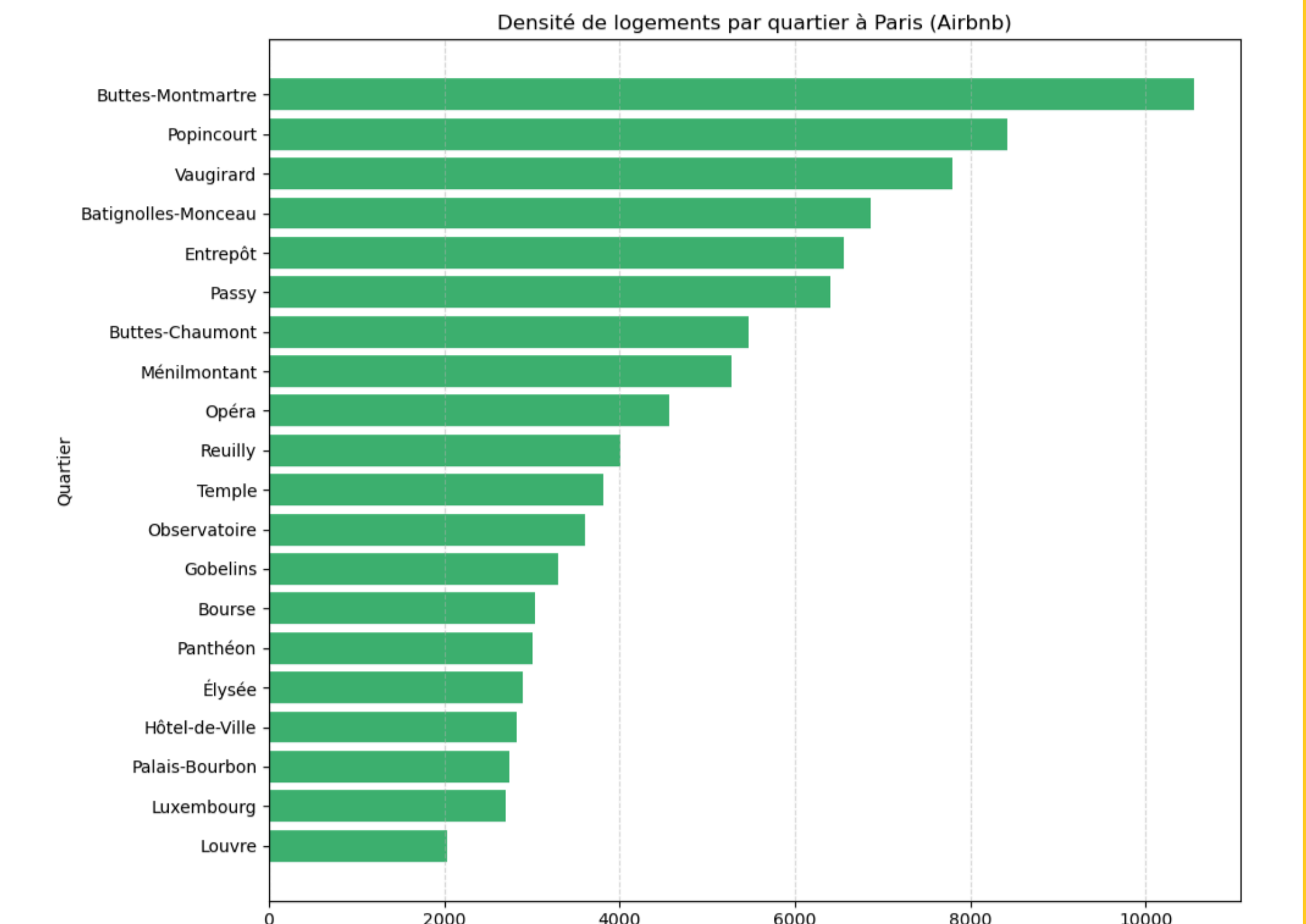
rows = list(cursor)
if not rows:
    raise ValueError("❌ Aucune donnée avec 'neighbourhood_cleansed' trouvée.")

# 3 Chargement dans Polars
df = pl.DataFrame(rows)

# 4 Comptage par quartier
agg = (
```

Requête ++ (4/5)

Calculer la médiane des nombre d'avis par catégorie d'hôte



Requête ++ (4/5)

Médiane des nombre d'avis par catégorie d'hôte

```
# 2 Extraction des champs nécessaires
cursor = collection.find(
    {
        "reviews_per_month": {"$exists": True, "$ne": None},
        "neighbourhood_cleansed": {"$exists": True, "$ne": None}
    },
    {
        "_id": 0,
        "reviews_per_month": 1,
        "neighbourhood_cleansed": 1
    }
)

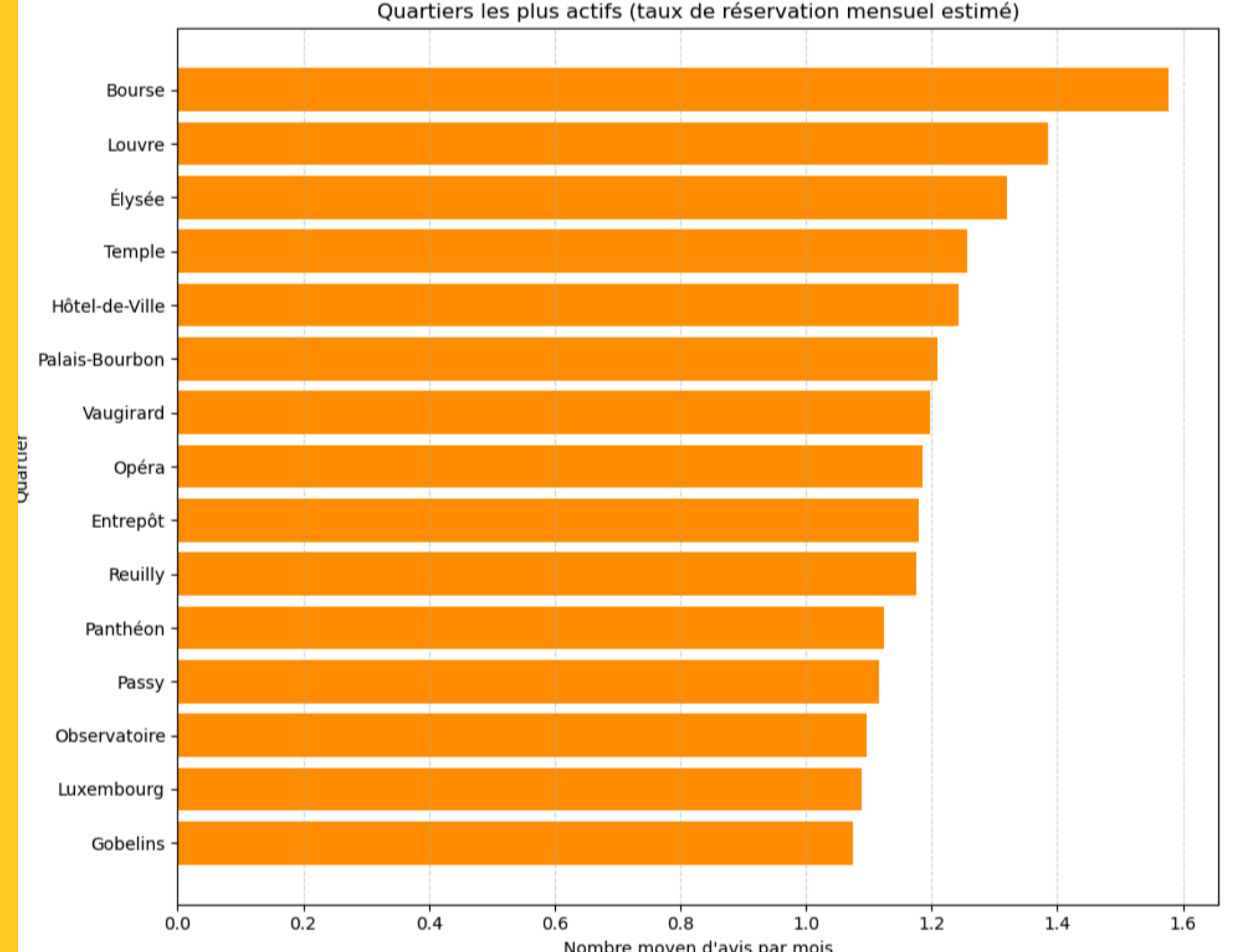
rows = list(cursor)
if not rows:
    raise ValueError("❌ Aucune donnée avec 'reviews_per_month' et 'neighbourhood_cleansed' trouvée.")

# 3 Chargement dans Polars
df = pl.DataFrame(rows)

# 4 Calcul du taux estimé par quartier
agg = (
    df
    .group_by("neighbourhood_cleansed")
    .agg([
        pl.mean("reviews_per_month").alias("avg_reviews_per_month"),
        pl.len().alias("listing_count")
    ])
    .sort("avg_reviews_per_month", descending=True)
)
```

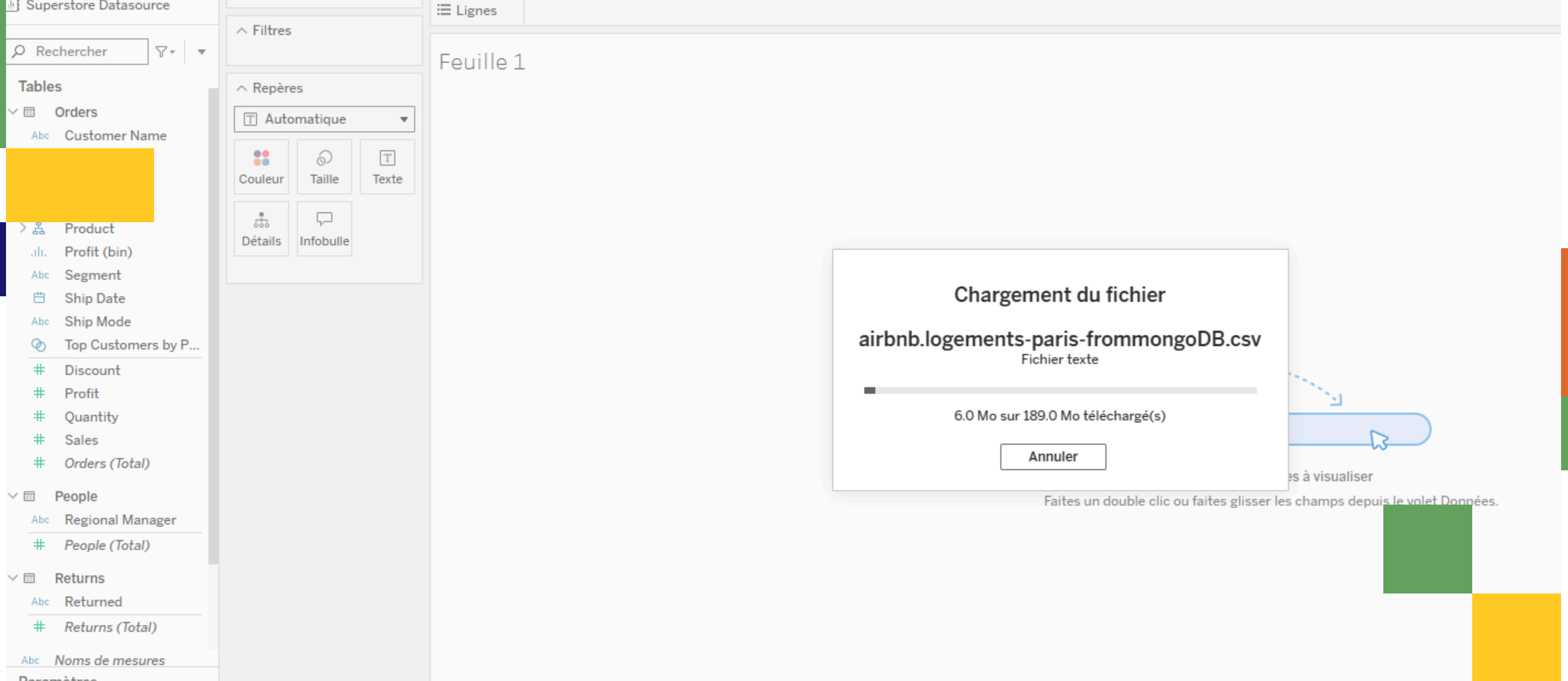
Requête ++ (5/5)

Identifier les quartiers avec le plus fort taux de réservation par mois



Requête ++ (5/5)

Identifier les quartiers avec le plus fort taux de réservation par mois



Utilisation de Tableau

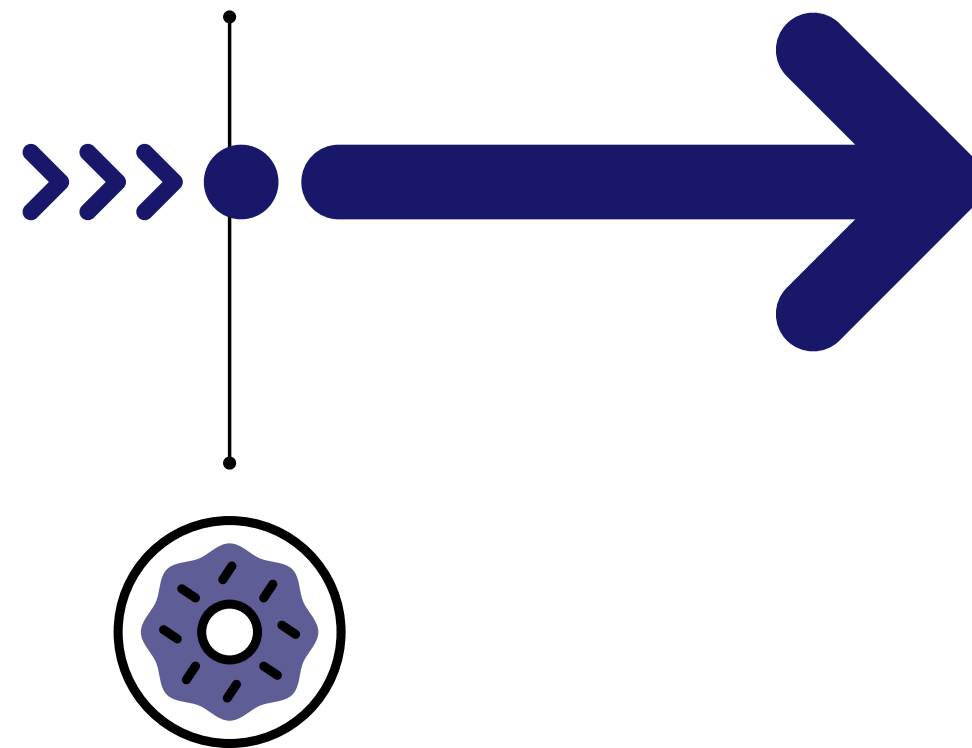
Téléchargement d'un export csv de la base Mongo DB

Répliquer et distribuer les données efficacement

Étapes claires pour la réplication et la distribution vers consommateurs et outils de BI

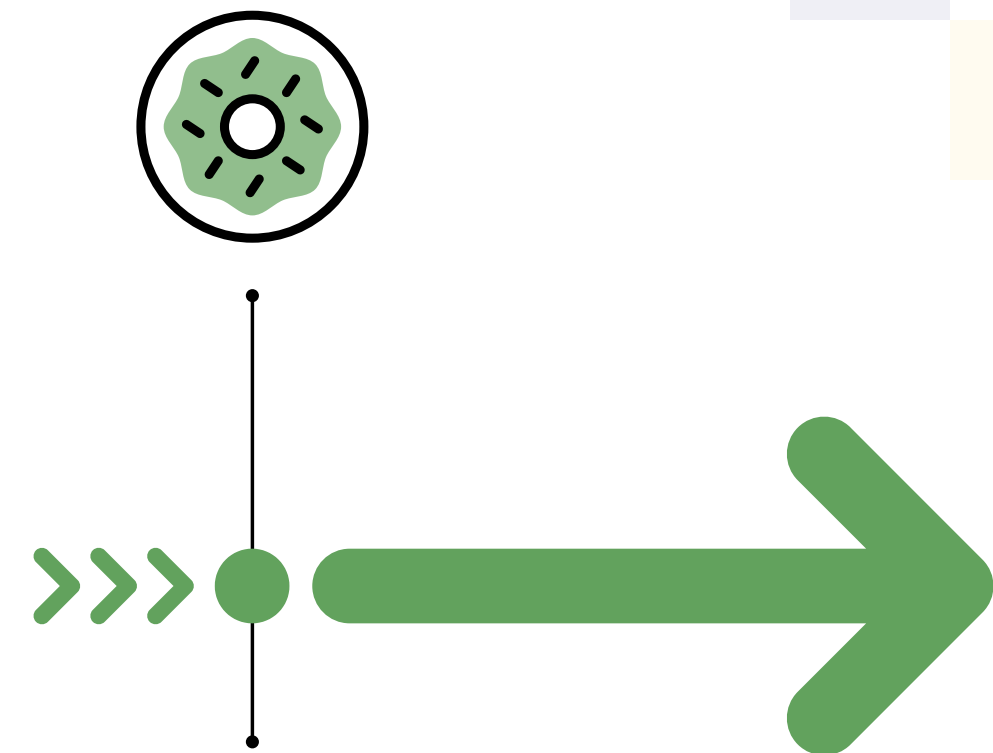
Répliquer les données

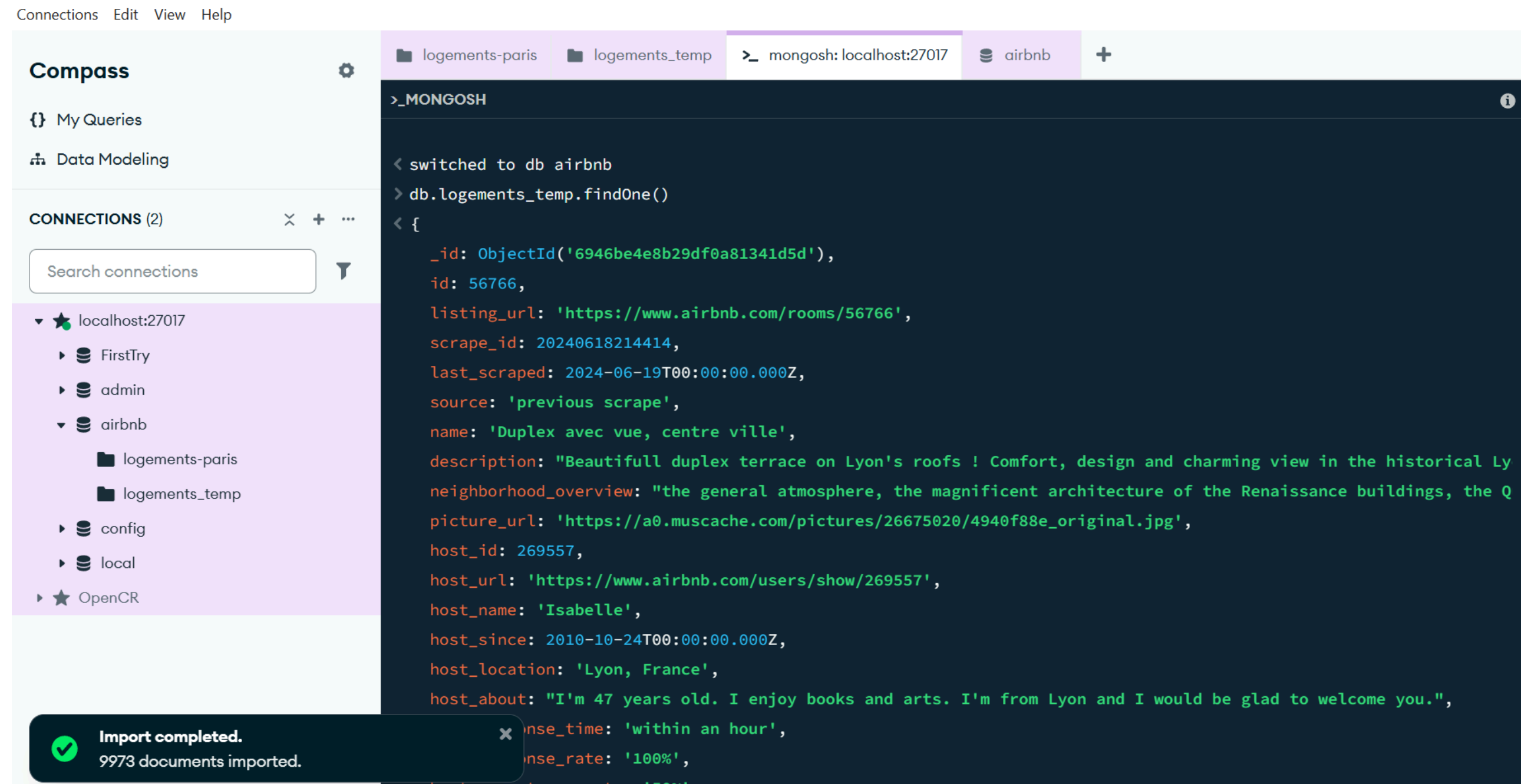
- Importation complète
- Spécifier une logique de réplication



Distribuer les données

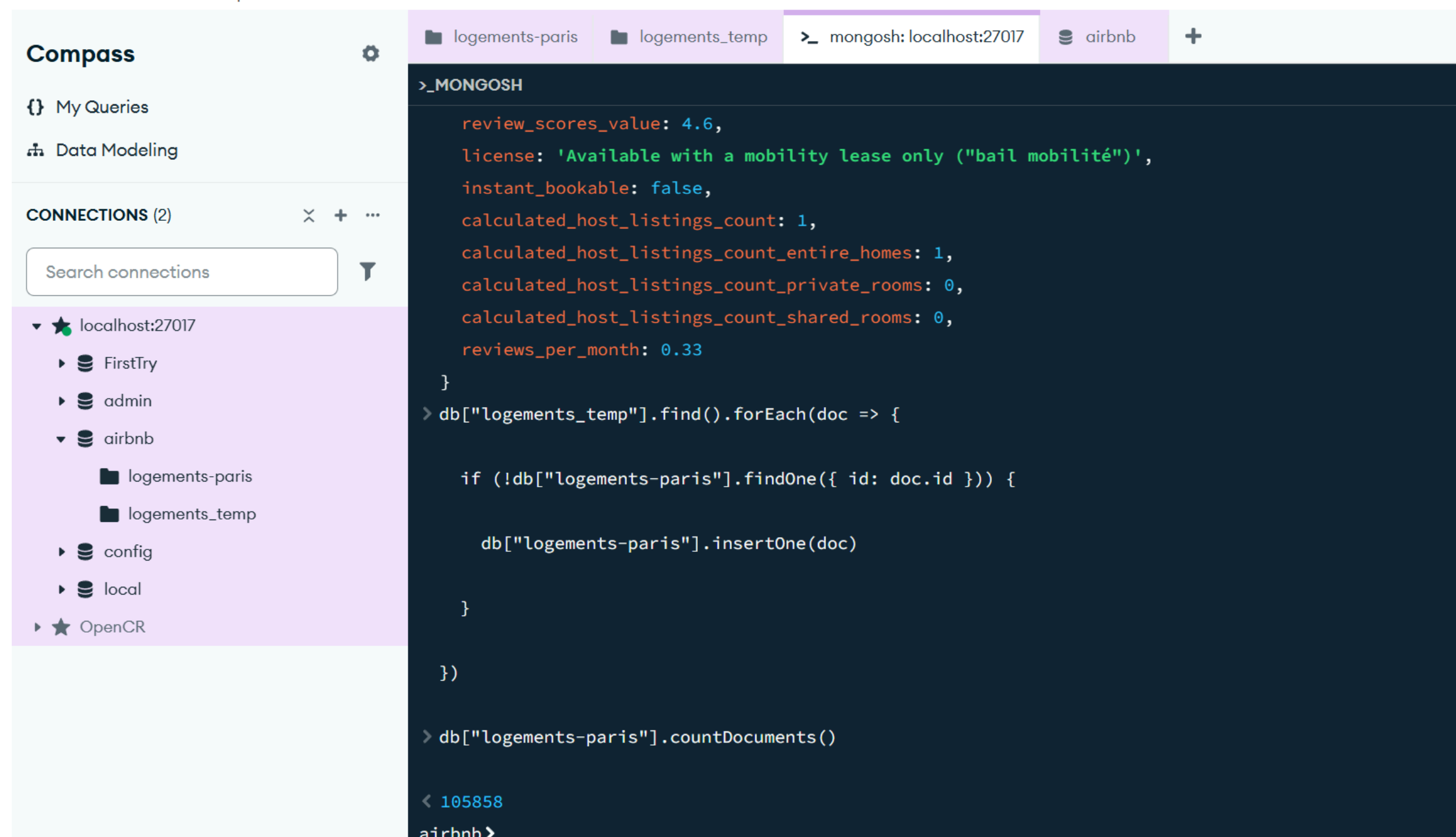
Distribuer les données — instructions étape 3 :
étapes détaillées pour la distribution vers
consommateurs et BI.





Import dans collection temporaire des données de Lyon

mongoimport --uri="mongodb://localhost:27017" --db airbnb --collection logements_temp --type csv --headerline --file listings_Lyon+(1).csv



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists several connections, with 'localhost:27017' expanded to show databases like 'logements-paris' and 'logements_temp'. The main panel displays a MongoDB Shell session with the following code:

```
>_MONGOSH

review_scores_value: 4.6,
license: 'Available with a mobility lease only ("bail mobilité")',
instant_bookable: false,
calculated_host_listings_count: 1,
calculated_host_listings_count_entire_homes: 1,
calculated_host_listings_count_private_rooms: 0,
calculated_host_listings_count_shared_rooms: 0,
reviews_per_month: 0.33
}
> db["logements_temp"].find().forEach(doc => {

  if (!db["logements-paris"].findOne({ id: doc.id })) {

    db["logements-paris"].insertOne(doc)

  }

})

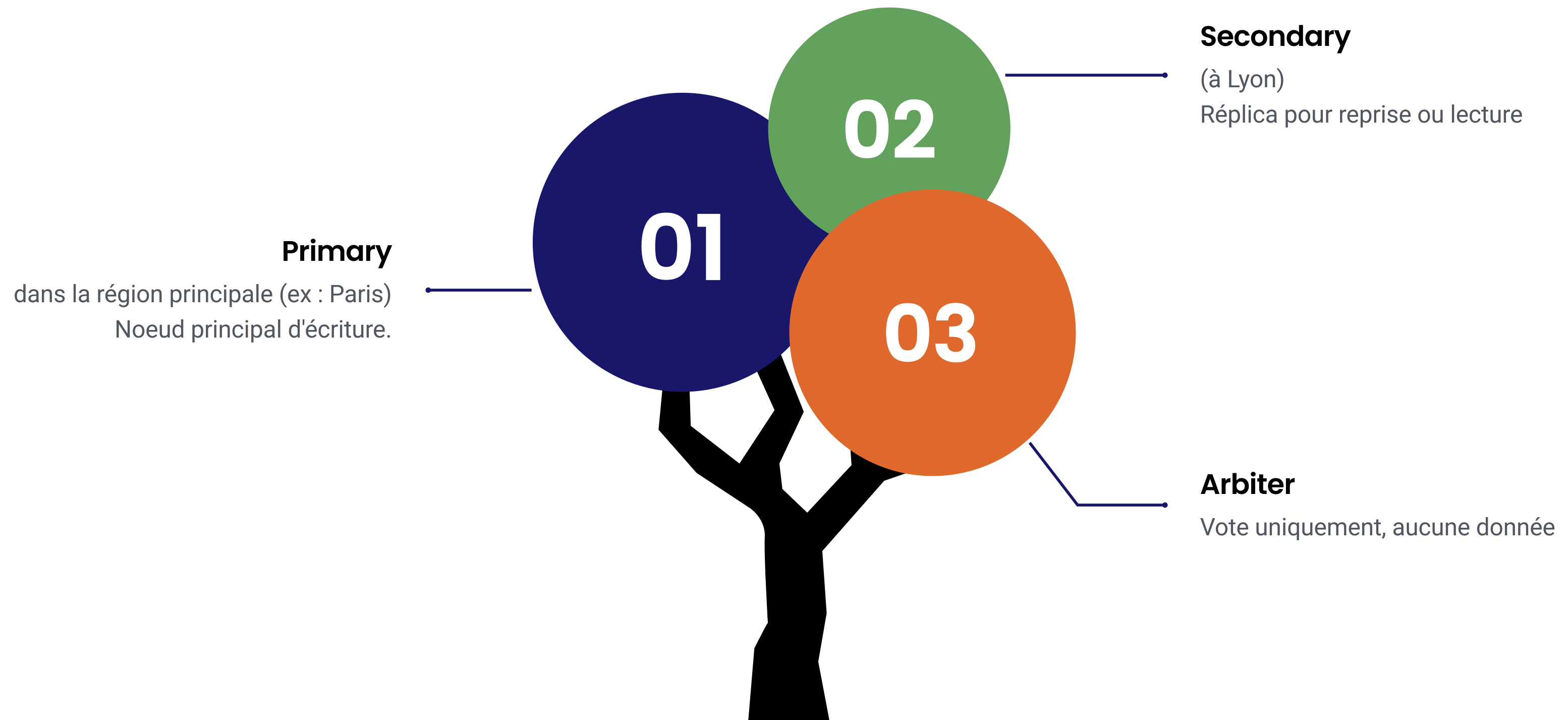
> db["logements-paris"].countDocuments()

< 105858
airbnb>
```

Insérer uniquement les nouveaux id (évite doublons)

```
db["logements_temp"].find().forEach(doc => { if (!db["logements-paris"].findOne({ id: doc.id })) { db["logements-paris"].insertOne(doc) } })
```

Replica Set réparti géographiquement



MongoDB Compass

Database name : airbnb Collection : logements-paris-lyon Storage Size :
106.82 MB **105858 documents mais 73262 seulement sur paris seul
et Lyon**

localhost:27017 > airbnb > logements-paris Open MongoDB shell

Documents 96K **Aggregations** Schema Indexes 1 Validation

✦ npmbre de documents des host location sur région parisienne comparer à ceux sur Lyon Generate Like Share

📁 ▾ **\$match** \$group \$unionWith Explain Export Run Options ▾

Untitled - modified SAVE + CREATE NEW EXPORT TO LANGUAGE PREVIEW STAGES TEXT WIZARD

```
1 [
2   {
3     $match: {
4       host_location: {
5         $regex: /paris/i
6       }
7     },
8   },
9   {
10    $group: {
11      _id: "Paris",
12      count: {
13        $sum: 1
14      }
15    }
16  },
17 ]
```

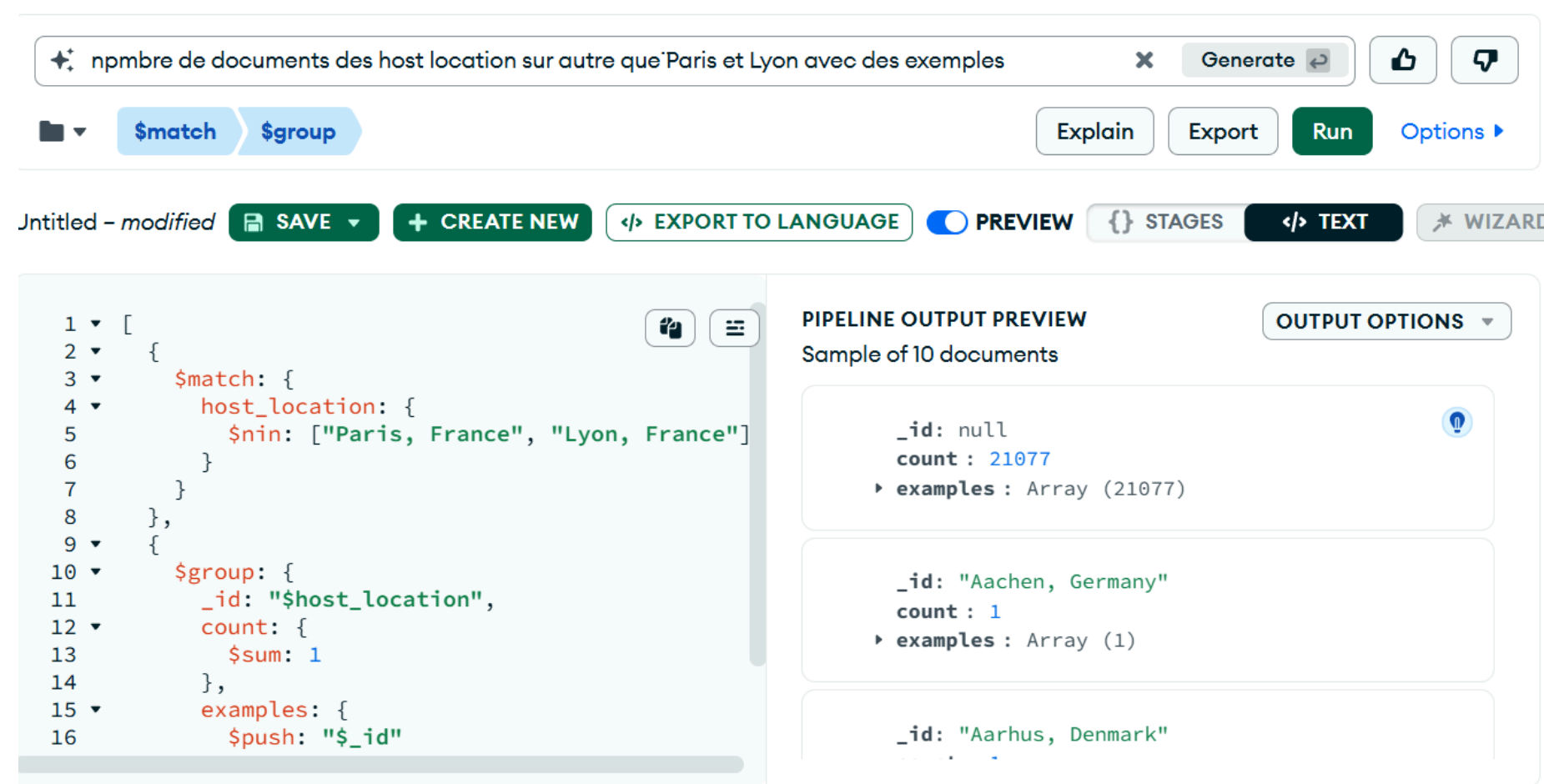
PIPELINE OUTPUT PREVIEW OUTPUT OPTIONS ▾
Sample of 2 documents

_id: "Paris"
count : 66287

_id: "Lyon"
count : 6975

MongoDB Compass

21077 documents ne concernent ni Paris ni Lyon



The image shows the MongoDB Compass web interface. At the top, a search bar contains the text "npmbre de documents des host location sur autre que Paris et Lyon avec des exemples". Below the search bar, there are tabs for "\$match" and "\$group", with "\$group" currently selected. To the right of the tabs are buttons for "Explain", "Export", "Run", and "Options". Below the tabs, there is a row of buttons: "Jntitled - modified", "SAVE", "CREATE NEW", "EXPORT TO LANGUAGE", "PREVIEW", "STAGES", "TEXT", and "WIZARD". The main area is split into two panels. The left panel shows a JSON query pipeline with 16 lines of code. The right panel, titled "PIPELINE OUTPUT PREVIEW", shows a sample of 10 documents. The first document has "_id: null" and "count: 21077". The second document has "_id: 'Aachen, Germany'" and "count: 1". The third document has "_id: 'Aarhus, Denmark'".

```
1 [
2   {
3     $match: {
4       host_location: {
5         $nin: ["Paris, France", "Lyon, France"]
6       }
7     },
8   },
9   {
10    $group: {
11      _id: "$host_location",
12      count: {
13        $sum: 1
14      },
15      examples: {
16        $push: "$_id"
```

PIPELINE OUTPUT PREVIEW
Sample of 10 documents

OUTPUT OPTIONS

_id: null
count: 21077
examples: Array (21077)

_id: "Aachen, Germany"
count: 1
examples: Array (1)

_id: "Aarhus, Denmark"

Pistes tests de réplication et de haute disponibilité

Vérifier réplication, contrôle d'accès
en lecture et comportement
d'arbiter

Test	Objectif	Ce qui est validé
✓ Test 1 : réplication	Que les données insérées sur le PRIMARY sont visibles sur le SECONDARY	+ INSERT sur PRIMARY → 👁️ READ sur SECONDARY
✓ Test 2 : lecture bloquée sur SECONDARY sans autorisation	Que les nœuds secondaires ne sont pas lisibles sans permission explicite	🛑 sécurité par défaut
✓ Test 3 : simulation de panne et vote de l'arbiter	Que l'arbiter joue son rôle pour élire un nouveau PRIMARY	⚙️ failover + quorum

Test 1a)

Le script shell : test-replication-manuel.ps1

```
$basePath = "C:\Users\karap\mongodb"

# Nodes definition
$nodes = @(
    @{ name = "PARIS";      port = 27017; folder = "rs-paris" },
    @{ name = "LYON";       port = 27018; folder = "rs-lyon" },
    @{ name = "STRASBOURG"; port = 27019; folder = "rs-strasbourg" }
)

# Create data folders if needed
foreach ($node in $nodes) {
    $path = Join-Path $basePath $node.folder
    if (-not (Test-Path $path)) {
        Write-Host "Creating folder $path"
        New-Item -ItemType Directory -Path $path | Out-Null
    }
}

# Start each mongod
foreach ($node in $nodes) {
    $dataPath = Join-Path $basePath $node.folder
    $logPath = Join-Path $dataPath "mongod.log"

    Write-Host "Starting $($node.name) on port $($node.port)"

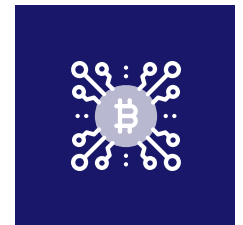
    Start-Process -FilePath $mongod `
        -ArgumentList @(
```



**Nodes
definition**



**Create
data
folders if
needed**



**Start
each
mongod**



Test a)

Vérification rapide des opérations de réplication

```
rsGeo [direct: secondary] test> rs.secondaryOk()
DeprecationWarning: .setSecondaryOk() is deprecated. Use .setReadPref("primaryPreferred") instead
Setting read preference from "primary" to "primaryPreferred"

rsGeo [direct: secondary] test> use testReplica
switched to db testReplica
rsGeo [direct: secondary] testReplica> db.demo.find().pretty()
...
[
  {
    _id: ObjectId('6948149a1bd5ef20a41e2621'),
    message: 'Bonjour depuis le PRIMARY F R ',
    createdAt: ISODate('2025-12-21T15:39:06.813Z')
  }
]
```



Lecture depuis SECONDARY 

Test b)

Valider qu'en cas de panne du PRIMARY (Paris, port 27017) Le SECONDARY (Lyon, 27018) devient automatiquement PRIMARY

```
rsGeo [direct: primary] test> rs.isMaster()
...
{
  topologyVersion: {
    processId: ObjectId('694926f687dbbad4caf777dc'),
    counter: Long('7')
  },
  hosts: [ 'localhost:27017', 'localhost:27018' ],
  arbiters: [ 'localhost:27019' ],
  setName: 'rsGeo',
  setVersion: 1,
  ismaster: true,
  secondary: false,
  primary: 'localhost:27018',
  me: 'localhost:27018',
  electionId: ObjectId('7fffffff000000000000000004'),
  lastWrite: {
    opTime: { ts: Timestamp({ t: 1766403252, i: 1 }), t: Long('4') },
    lastWriteDate: ISODate('2025-12-22T11:34:12.000Z'),
    majorityOpTime: { ts: Timestamp({ t: 1766403194, i: 1 }), t: Long('3') },
    majorityWriteDate: ISODate('2025-12-22T11:33:14.000Z')
  },
  maxBsonObjectSize: 16777216,
  maxMessageSizeBytes: 48000000,
  maxWriteBatchSize: 100000,
  localTime: ISODate('2025-12-22T11:34:19.141Z'),
  logicalSessionTimeoutMinutes: 30,
  connectionId: 61,
  ...
}
```



Panne simulée
du PRIMARY ✓



Election
automatique ✓

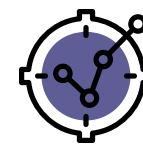


SECONDARY
devient PRIMARY
✓

Stratégie de distribution MongoDB: optimiser données et charge

Quand et pourquoi sharder (scalabilité, performance et résilience..)

01



But général

Distribuer intelligemment les **données et la charge de travail** (écritures + lectures) entre plusieurs serveurs (shards)

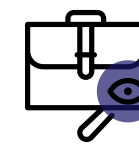
02



Objectifs clés

-  **Scalabilité horizontale** : Répartir la donnée entre plusieurs machines
-  **Performances** : Réduire le nombre de documents lus/analysés par serveur
-  **Haute disponibilité** : Chaque shard est un Replica Set
-  **Isolation logique** : Permet de séparer selon des règles métiers (ex : zones, rôles, entités...)

03



Contexte spécifique

- IL n'y a **pas qu'un seul entrepôt géographique**, mais une **structure binodale** :
-  Paris : majorité des logements dans cette ville
-  Lyon : autre base importante, autre siège de l'association
-  Objectif à moyen terme : distribuer la **charge**, la **donnée**, et garantir **tolérance aux pannes** + **gestion localisée des données**

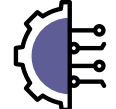
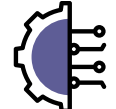
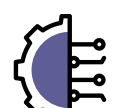
Sharding par clé : ville

Représenter la structure réelle, extensible, ciblé, logique

Shard	Localisation physique	Ville ciblée
Shard 1 (shardParis)	Serveur MongoDB à Paris	host-location: "Paris, .."
Shard 2 (shardLyon)	Serveur MongoDB à Lyon	host-location: "Lyon, .."
Shard 3 (shardBonus)	Hors Paris et Lyon	host-location: Hors Paris et Lyon
Shard 3' (shardCloud)	Cloud (ou future ville.autre) ou autre	Pour extensions ou backup général

Architecture proposée pour le sharding MongoDB

Détails des composants, ports et dossiers de données

Composant	Rôle	Port	Dossier de données
config1	Config server	26001	 C:\Users\karap\mongodb\cfg1
config2	Config server	26002	 C:\Users\karap\mongodb\cfg2
config3	Config server	26003	 C:\Users\karap\mongodb\cfg3
shard1 (Paris)	Shard server (RS)	27017	 C:\Users\karap\mongodb\shard1
shard2 (Lyon)	Shard server (RS)	27018	 C:\Users\karap\mongodb\shard2
shard3 (Hors Paris et Lyon)	Shard server (RS)	27019	 C:\Users\karap\mongodb\shard3
mongos	Routeur mongos	27020	 pas de data

6 étape pour configurer un cluster MongoDB

Démarrage des config servers, shards, mongos et activation du sharding



Étape 1 — Crée les dossiers

**Étape 2 —
Démarre les
Config Servers**

**Étape 3 —
Démarrer les
shards**

**Étape 4 —
Démarrer le
mongos
(routeur)**

**Étape 5 —
Connexion à
mongos**

**Étape 6 — Activer
le sharding**



Cluster sharded prêt à l'emploi

init-sharded-cluster.ps1

Pascal Duval & chat GPT

```
# ✅ Config
$mongod = "C:\Program Files\MongoDB\Server\8.0\bin\mongod.exe"
$mongosh = "C:\Users\karap\Downloads\mongosh-2.5.10-win32-x64\mongosh-2.5.10-win32-x64\bin\mongosh.exe"
$baseDir = "C:\Users\karap\mongodb"
```

```
# ✅ Nettoyage
Write-Host "🔧 Nettoyage en cours..." -ForegroundColor Yellow
Stop-Process -Name mongod -Force -ErrorAction SilentlyContinue
Stop-Process -Name mongos -Force -ErrorAction SilentlyContinue
Start-Sleep -Seconds 2
```

```
# ✅ Dossiers attendus
$dirs = @("shard1", "shard2", "shard3", "cfg1", "cfg2", "cfg3")
foreach ($dir in $dirs) {
    $full = Join-Path $baseDir $dir
    if (-not (Test-Path $full)) {
        New-Item -ItemType Directory -Path $full | Out-Null
    }
}
```

```
# ✅ Lancement des Shards
$shards = @(27017, 27018, 27019)
for ($i = 0; $i -lt $shards.Length; $i++) {
    $port = $shards[$i]
    $folder = Join-Path $baseDir "shard$($i+1)"
    Start-Process -FilePath $mongod -ArgumentList @(
        "--shardsvr",
        | "--replSet", "shard$($i+1)Repl",
        "--port", "$port",
        "--dbpath", "`"$folder`"",
        "--bind_ip", "localhost",
        "--logpath", "`"$folder\mongod.log`"",
        "--logappend"
    ) -WindowStyle Hidden
    Write-Host "✅ Shard lancé : shard$($i+1)Repl sur port $port"
}
```

```
# ✅ Lancement des Config Servers
$cfgPorts = @(26001, 26002, 26003)
for ($i = 0; $i -lt $cfgPorts.Length; $i++) {
    $port = $cfgPorts[$i]
    $folder = Join-Path $baseDir "cfg$($i+1)"
    Start-Process -FilePath $mongod -ArgumentList @(
        "--configsvr",
        "--replset", "cfgRepl",
        "--port", "$port",
        "--dbpath", "`"$folder`"",
        "--bind_ip", "localhost",
        "--logpath", "`"$folder\mongod.log`"",
        "--logappend"
    ) -WindowStyle Hidden
    Write-Host "✅ Config Server $i lancé sur port $port"
}
```

```
Start-Sleep -Seconds 10
```

```
# ✅ Lancer mongos
$mongosLog = "$baseDir\mongos.log"
Start-Process -FilePath "$mongod\..\mongos.exe" -ArgumentList @(
    "--configdb", "cfgRepl/localhost:26001,localhost:26002,localhost:26003",
    "--port", "27000",
    "--bind_ip", "localhost",
    "--logpath", "`"$mongosLog`"",
    "--logappend"
) -WindowStyle Hidden
Write-Host "✅ Routeur mongos lancé sur port 27000"
```

```
Start-Sleep -Seconds 10
```

📌 Initialisation des Replica Sets

```
Write-Host "🌀 Initialisation des Replica Sets..." -ForegroundColor Cyan
```

```
& $mongosh --port 27017 --eval "rs.initiate({ _id: 'shard1Repl', members: [{ _id: 0, host: 'localhost:27017' }] })"
& $mongosh --port 27018 --eval "rs.initiate({ _id: 'shard2Repl', members: [{ _id: 0, host: 'localhost:27018' }] })"
& $mongosh --port 27019 --eval "rs.initiate({ _id: 'shard3Repl', members: [{ _id: 0, host: 'localhost:27019' }] })"

& $mongosh --port 26001 --eval "rs.initiate({
  _id: 'cfgRepl',
  configsvr: true,
  members: [
    { _id: 0, host: 'localhost:26001' },
    { _id: 1, host: 'localhost:26002' },
    { _id: 2, host: 'localhost:26003' }
  ]
})"
```

```
Start-Sleep -Seconds 10
```

✅ Ajout des Shards

```
Write-Host "✚ Ajout des shards au cluster..." -ForegroundColor Cyan
```

```
& $mongosh --port 27000 --eval "sh.addShard('shard1Repl/localhost:27017')"
& $mongosh --port 27000 --eval "sh.addShard('shard2Repl/localhost:27018')"
& $mongosh --port 27000 --eval "sh.addShard('shard3Repl/localhost:27019')"
```

✅ État du cluster

```
Write-Host "`n🌈 État du cluster :" -ForegroundColor Cyan
```

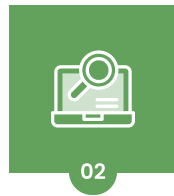
```
& $mongosh --port 27000 --eval "sh.status()"
```

Répartir une collection entre trois shards



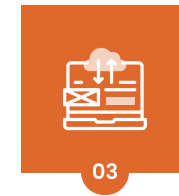
Objectif de routage

host_location "Paris, France" -> shard1Repl;
"Lyon, France" -> shard2Repl;
autres villes -> shard3Repl



Indexer la clé

Créer un index sur la clé {
host_location: 1 }
(obligatoire)



Activer le sharding

Activer le sharding sur la
base airbnbShard
(obligatoire)



Finaliser

déplacer les chunks
manuellement

État du sharding: vue d'ensemble rapide

sh.status() sur & "C:\Users\karap\Downloads\mongosh-2.5.10-win32-x64\mongosh-2.5.10-win32-x64\bin\mongosh.exe" --port 27000

Élément	État actuel	Commentaire
Shards enregistrés	✔ Oui	Tous bien visibles
Clé de sharding	✔ { host_location: 1 }	Correct
Données réparties	⚠ shard3 vide	Besoin d'ajustement
Chunk Paris	✔ sur shard1	OK
Chunk Lyon	✔ sur shard2	OK
Chunk “reste du monde”	✗ non présent sur shard3	À corriger
Index nécessaire	✔ déjà créé pour sharding	Aucun souci

Répartition finale des shards pour logementsParisLyon

db.logementsParisLyon.getShardDistribution() sur & "C:\Users\karap\Downloads\mongosh-2.5.10-win32-x64\mongosh-2.5.10-win32-x64\bin\mongosh.exe" --port 27000

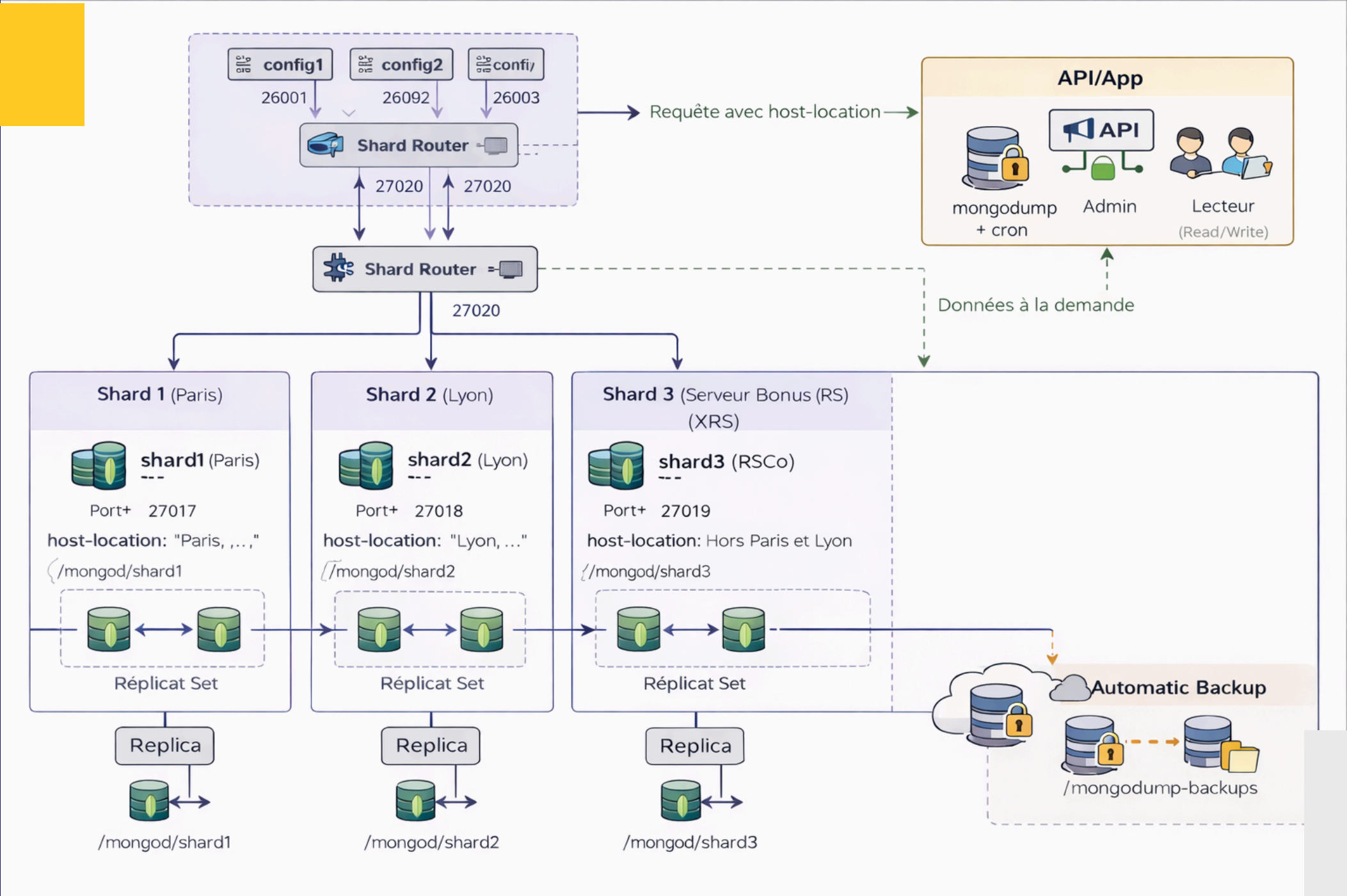
Shard	Donnée (data)	Documents (docs)	Chunks	Estimated docs per chunk
shard3Repl at shard3Repl/localhost:27019	90.58MiB	27396	1	 27396
shard2Repl at shard2Repl/localhost:27018	29.51MiB	36307	1	 36307
shard1Repl at shard1Repl/localhost:27017	238.88MiB	69556	1	 69556
Shard shard3Repl (pourcentages)	25.23 % data	20.55 % docs in cluster	3KiB avg obj size on shard	 hors Paris et Lyon
Shard shard2Repl (pourcentages)	8.22 % data	27.24 % docs in cluster	3KiB avg obj size on shard	 Lyon
Shard shard1Repl (pourcentages)	66.54 % data	52.19 % docs in cluster	3KiB avg obj size on shard	 Paris

Architecture finale

Serveur, usager et liens entre replicats

API applicative

Elle ramène en fonction du host_location de la demande les données es shards concernés





Concevoir et analyser une base de données NoSQL

"... dans la perspective d'un rapport qui doit étayer l'effet "Jeux Olympiques 2024" sur l'offre de logements à Paris ..."



Pascal DUVAL

AI/ML Engineer